
Neural Networks

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

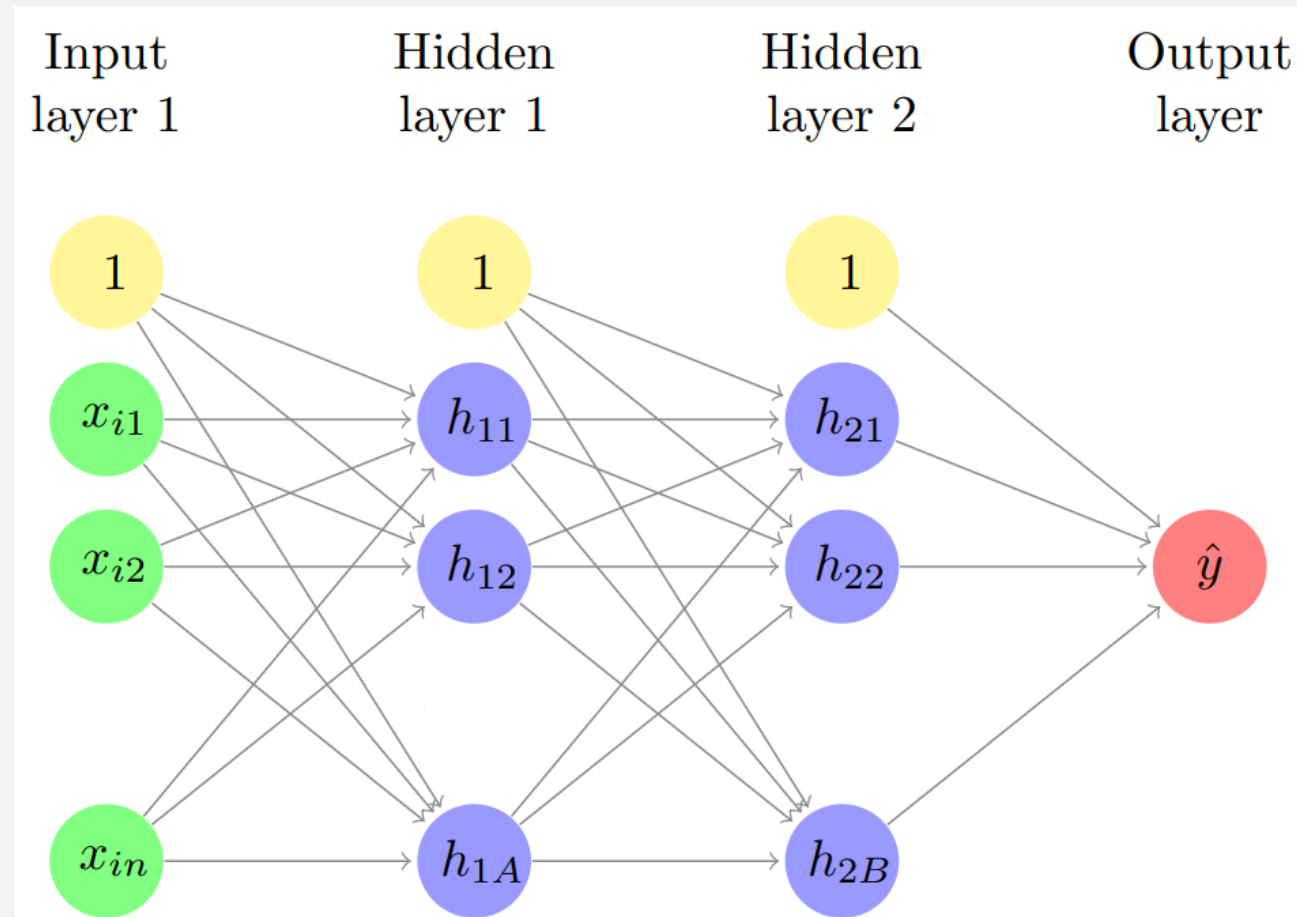
This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Neural Networks – Intro
- Single Layer Networks
 - Input layer, output layer
 - Linear regression neural network
 - Logistic regression neural network
 - Backpropagation: Learning single layer weights by minimizing loss function
- Multilayer Networks
 - Input layer, hidden layers, output layer
 - Feedforward process
 - Backpropagation: Learning multilayer weights by minimizing loss function
- Summary

Overview [1]

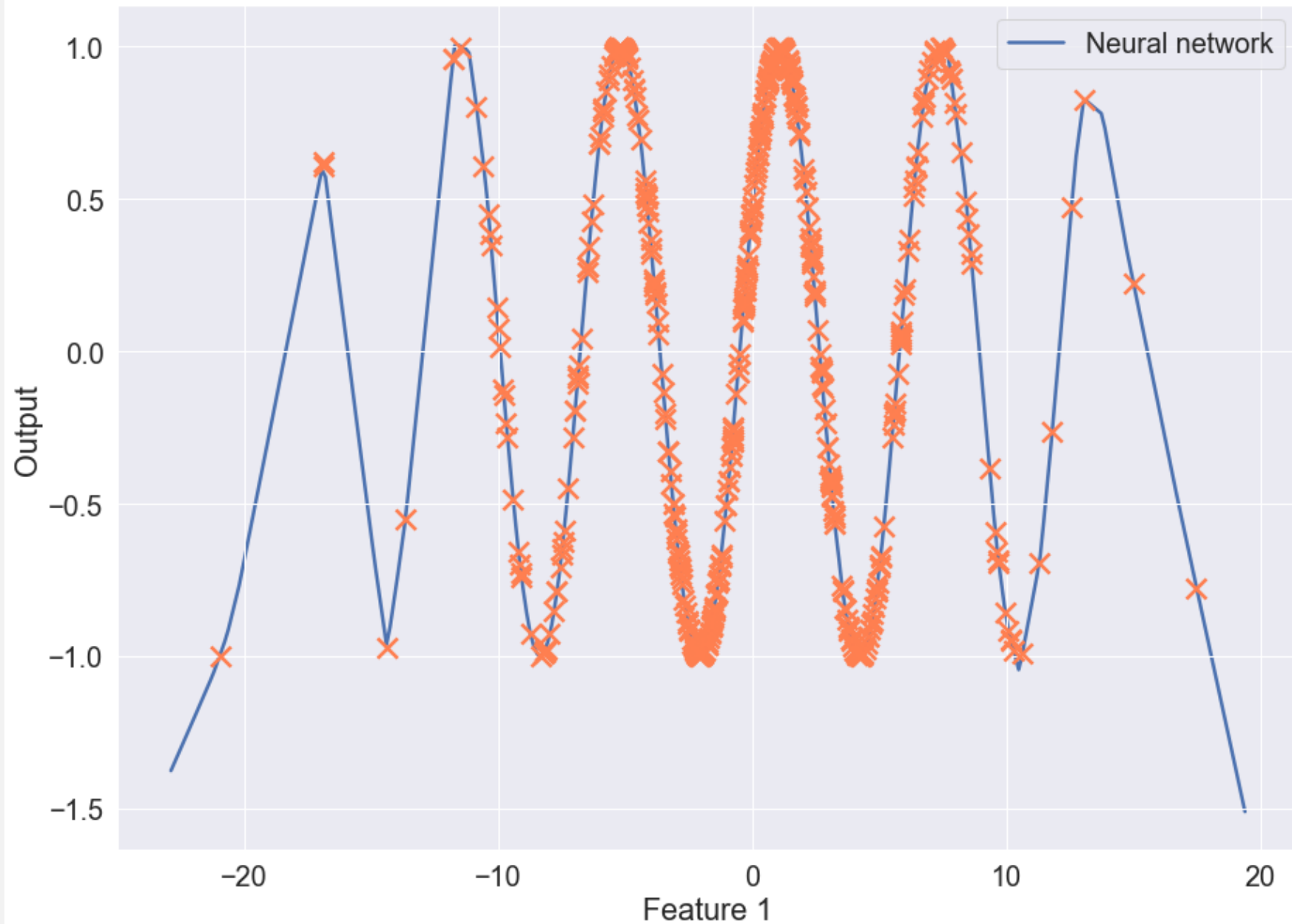
- Inspired by biological networks in the brain. Brain neurons are interconnected via synapses. These neurons communicate with each via these synapse connections.
- A neural network is represented as a directed graph. Nodes (neurons) point to other nodes via arrows. The arrows indicate the direction of data flow (synapses).



Overview [2]

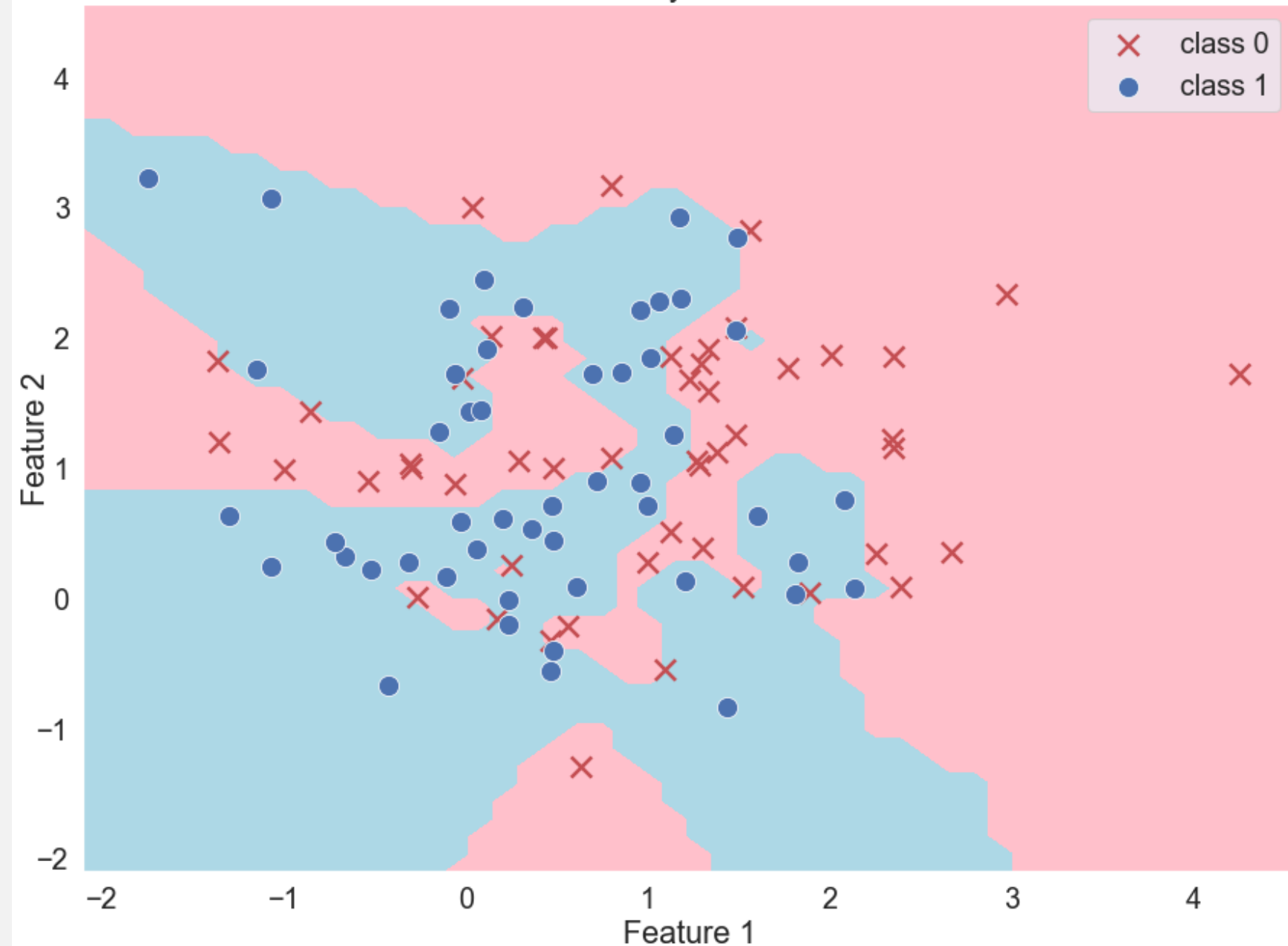
- Very powerful ML algorithms capable of approximating complex functions. Thus, with lots of data, leads to a powerful trained model.
- Due to the internet, advancements in data centers, advancements in machines (particularly GPU devices), this has lead to resurgence of neural networks since the 2000s.
- Popular types of neural networks: fully-connected (FCN), convolutional (CNN), and recurrent (RNN).
- FCNs are suitable for the type of *tabular* data we've seen so far in class. That is, an instance is represented by a vector of n features.
- CCNs are suitable when the features of an instance has local spatial-dependency. For example: the instance is a 2D image in which case the pixels that are near each other have local dependency. Similarly, if your instance is a 3D object.
- RNNs are suitable when features of an instance have temporal-dependency. For example, in audio data, each signal at some time t depends on signals at times before t . Another example is text, where the next word prediction depends on previous words.
- Our focus will be on FCNs.

- See the one-feature regression dataset. A neural network was fit to this training data.
- Note that the network takes in feature 1 as input. So, we did not add any polynomial transformations of feature 1.
- Neural networks are capable of learning complex functions without the need to transform the input features beforehand like with polynomial linear regression.



Neural network fit for a binary classification two-feature task

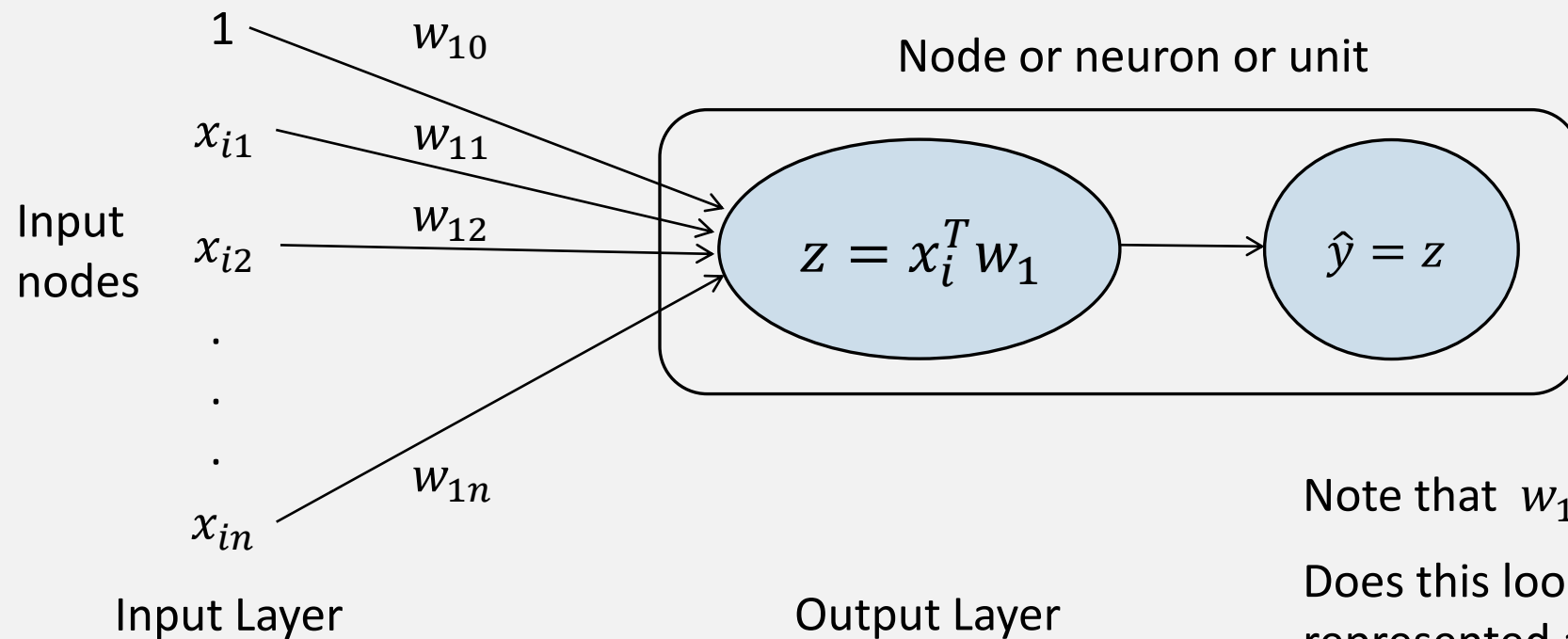
- Two-feature binary classification dataset. A neural network was fit to this training data.
- Note that the network takes in the two features as input. We did not add any polynomial transformations of these features.



Single Layer Neural Networks

Single layer NN formalism [1]

- The main components of a neural network are: **nodes** also called neurons or units, and **weights** also called connections.
- In a single layer NN, we only have the input layer and the output layer. The input layer represents the input features, and the output layer represents the output. For a regression task, this is illustrated below:

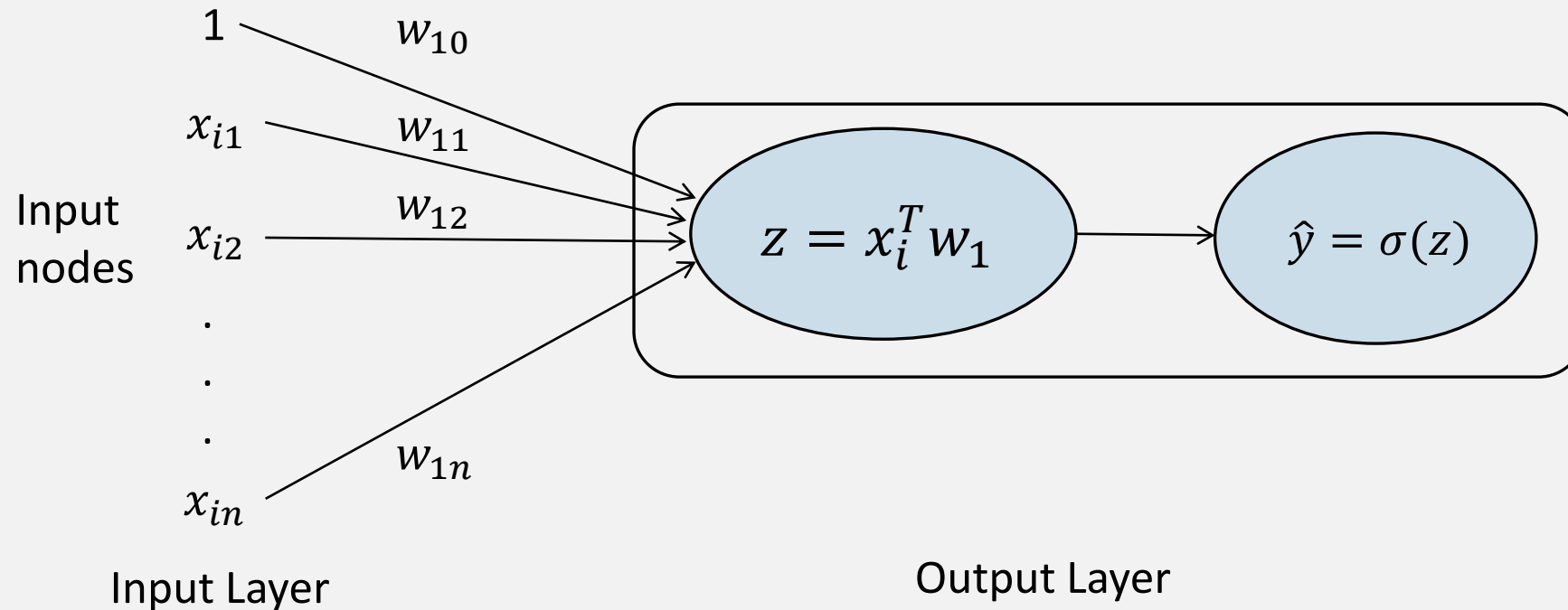


Note that $w_1 = [w_{10}, w_{11}, w_{12}, \dots, w_{1n}]$

Does this look familiar? Yes, it is linear regression represented as a neural network.

Single layer NN formalism [2]

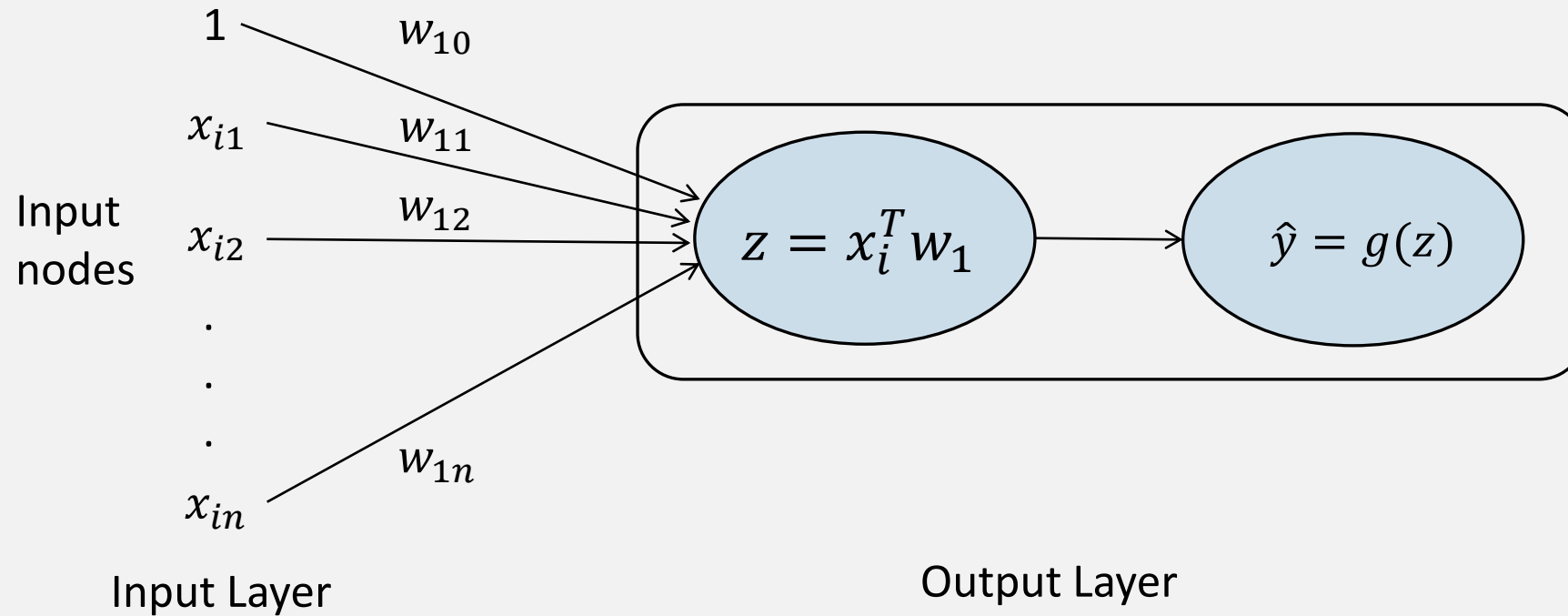
- For a binary classification task, the output node becomes:



Does this look familiar? Yes, it is logistic regression represented as a neural network.

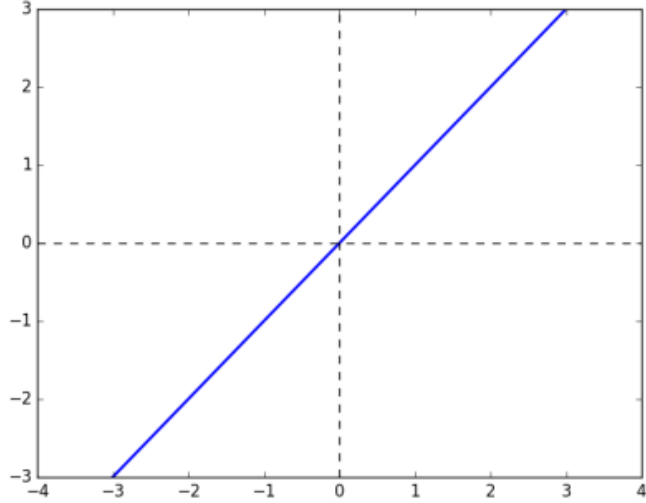
Single layer NN formalism [3]

- In general, you can apply any function g for a node:



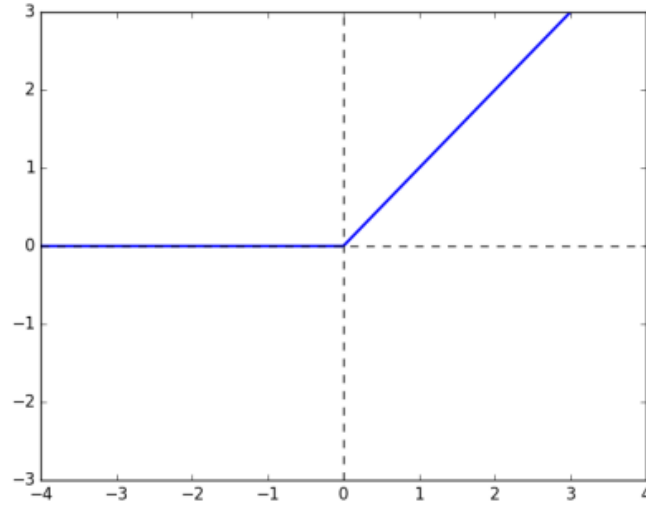
- It just needs to be differentiable so that we can define a loss and perform gradient descent to update the weights.
- The function applied at the node is called an **activation function**.

Common activation function choices [1]



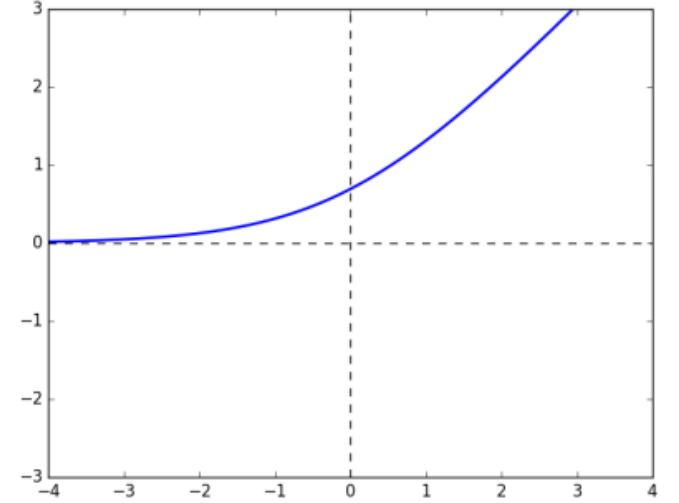
Linear

$$y = z$$



**Rectified Linear Unit
(ReLU)**

$$y = \max(0, z)$$

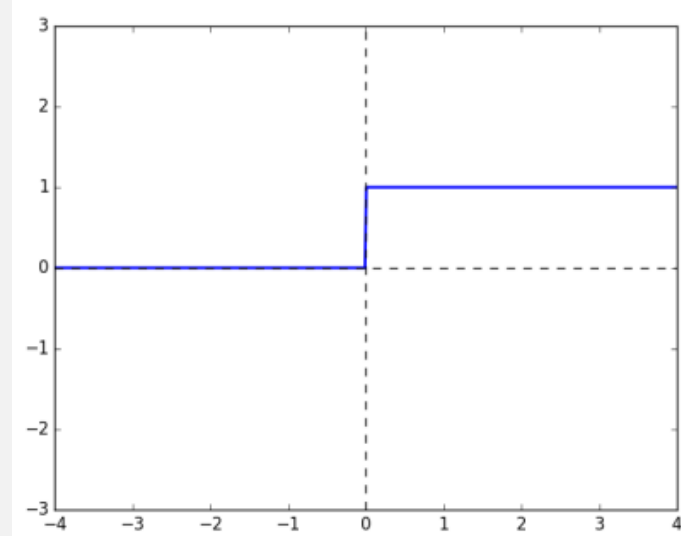


Soft ReLU

$$y = \log 1 + e^z$$

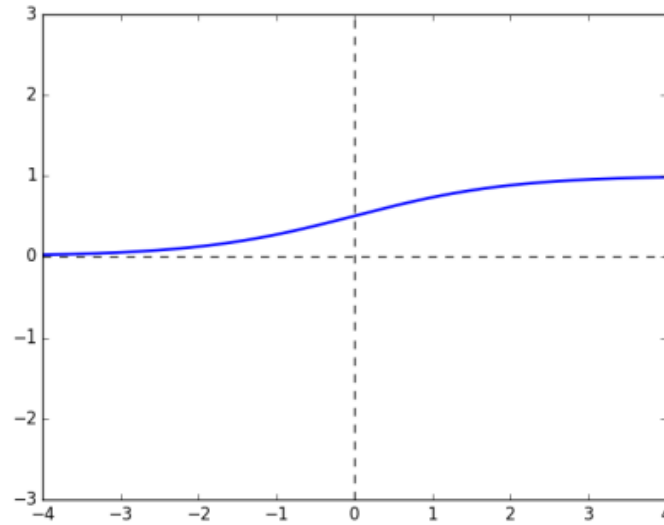
Image from: [link](#)

Common activation function choices [2]



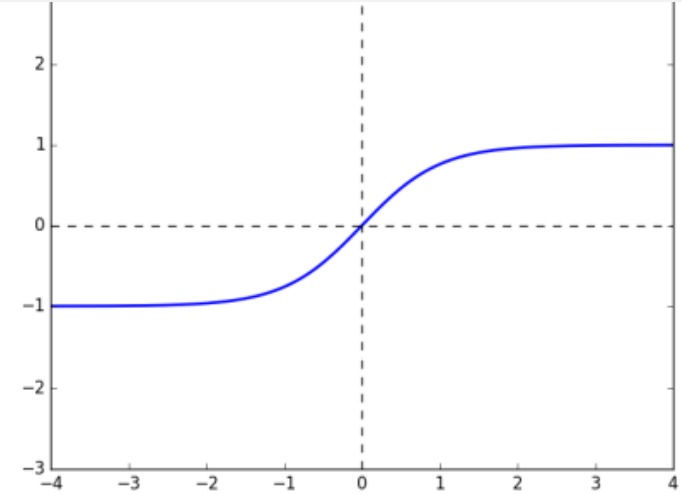
Hard Threshold

$$y = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$



Logistic

$$y = \frac{1}{1 + e^{-z}}$$



Hyperbolic Tangent (tanh)

$$y = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Image from: [link](#)

Single layer NN formalism [4]

- To train the network on training data and learn the weights, define a loss function and perform gradient descent.
- Common to use the squared error for regression tasks, just like in linear regression.
- Common to use the logloss (crossentropy) for classification tasks, just like in logistic regression.
- For a single layer network, compute the gradient of the loss function with respect to the weights, then update the weights. Similar to linear and logistic regression.

$$\hat{w}_1 := \hat{w}_1 - \alpha \frac{\partial \text{loss}(\hat{w}_1)}{\partial w_1}$$

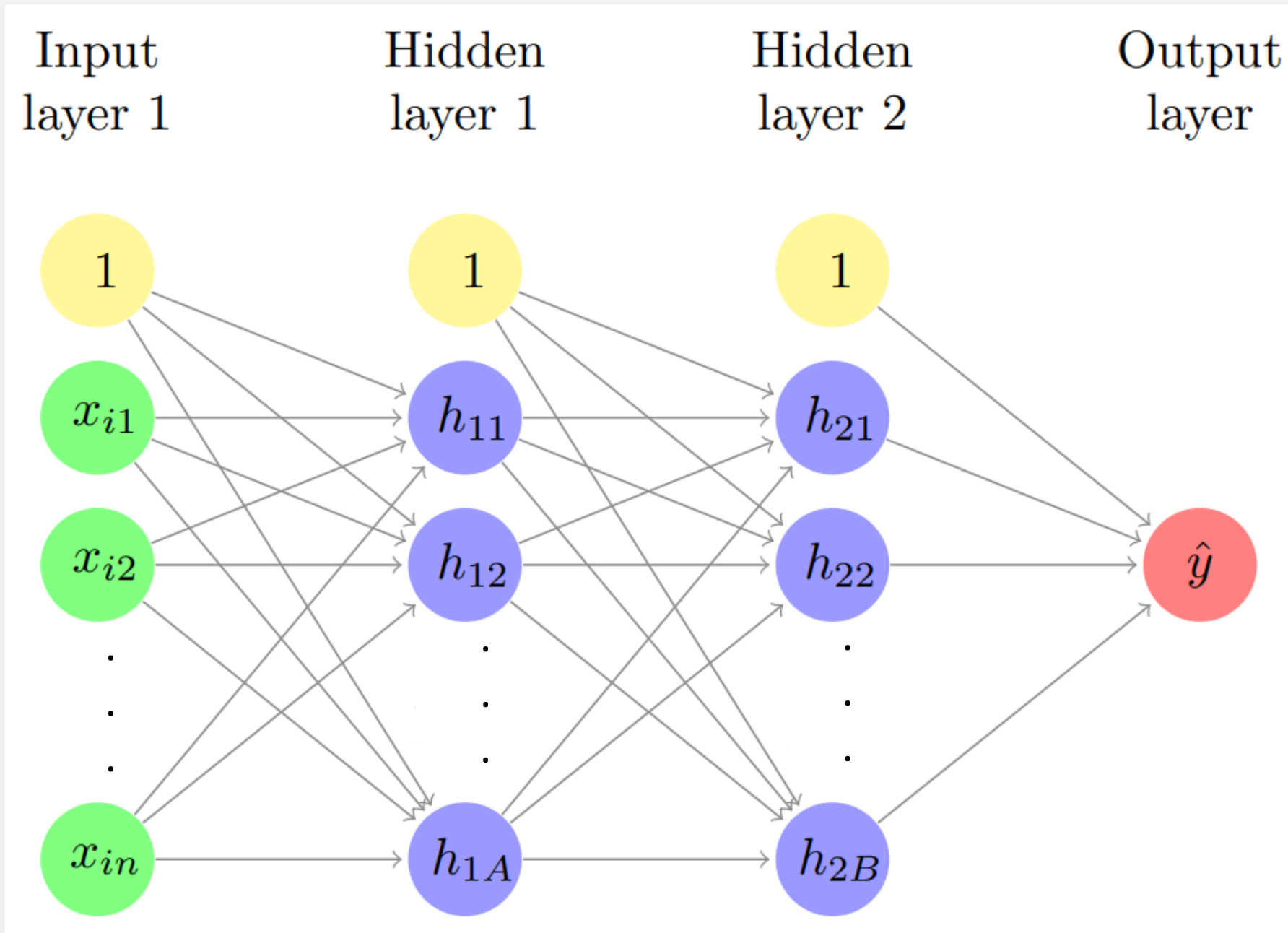
Or more explicitly, update each weight element:

$$\hat{w}_{1j} := \hat{w}_{1j} - \alpha \frac{\partial \text{loss}(\hat{w}_1)}{\partial w_{1j}}$$

Multilayer Neural Networks

Multilayer NN [1]

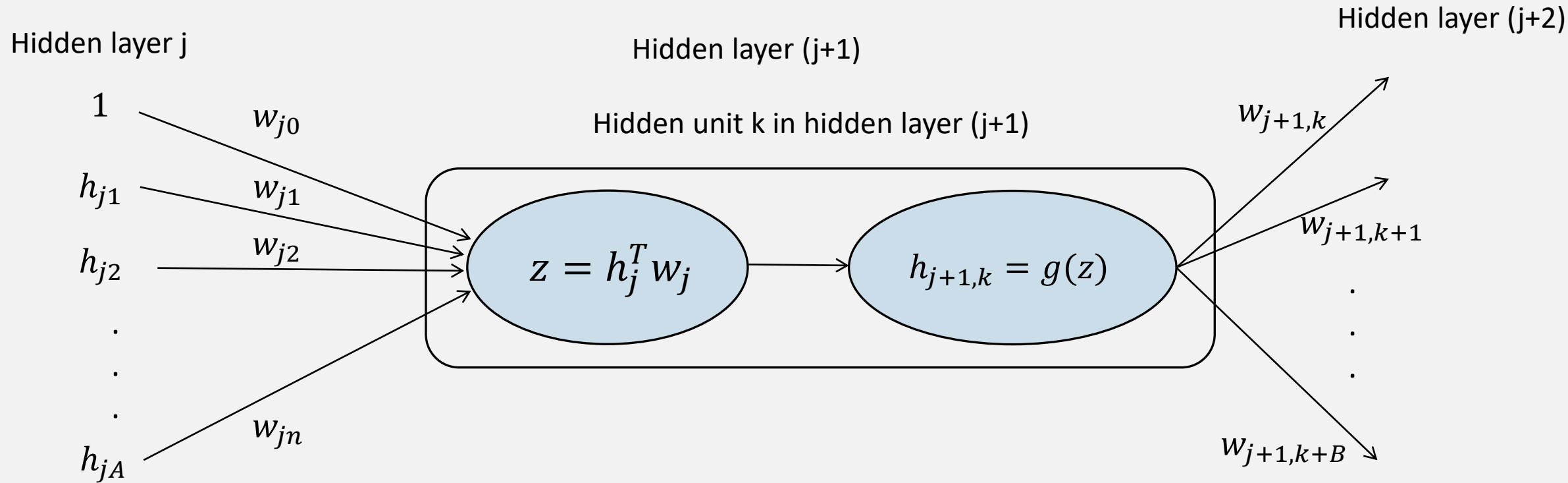
- A single layer network is not very powerful. It is just ordinary linear or logistic regression. What can we do to increase its complexity?
- Add intermediate layers between the input layer and output layer. These intermediate layers are called **Hidden Layers**.



- This is a multilayer network with 2 hidden layers.
- Each arrow represents a weight value that needs to be learned via training.
- Each hidden layer contains nodes that take in the weighted sum from the previous layer.
- Notice that hidden layers can have different number of units.
- The hidden units perform an activation function like sigmoid or ReLu.
- The final output layer is the prediction.
- Deep learning -> having many hidden layers.

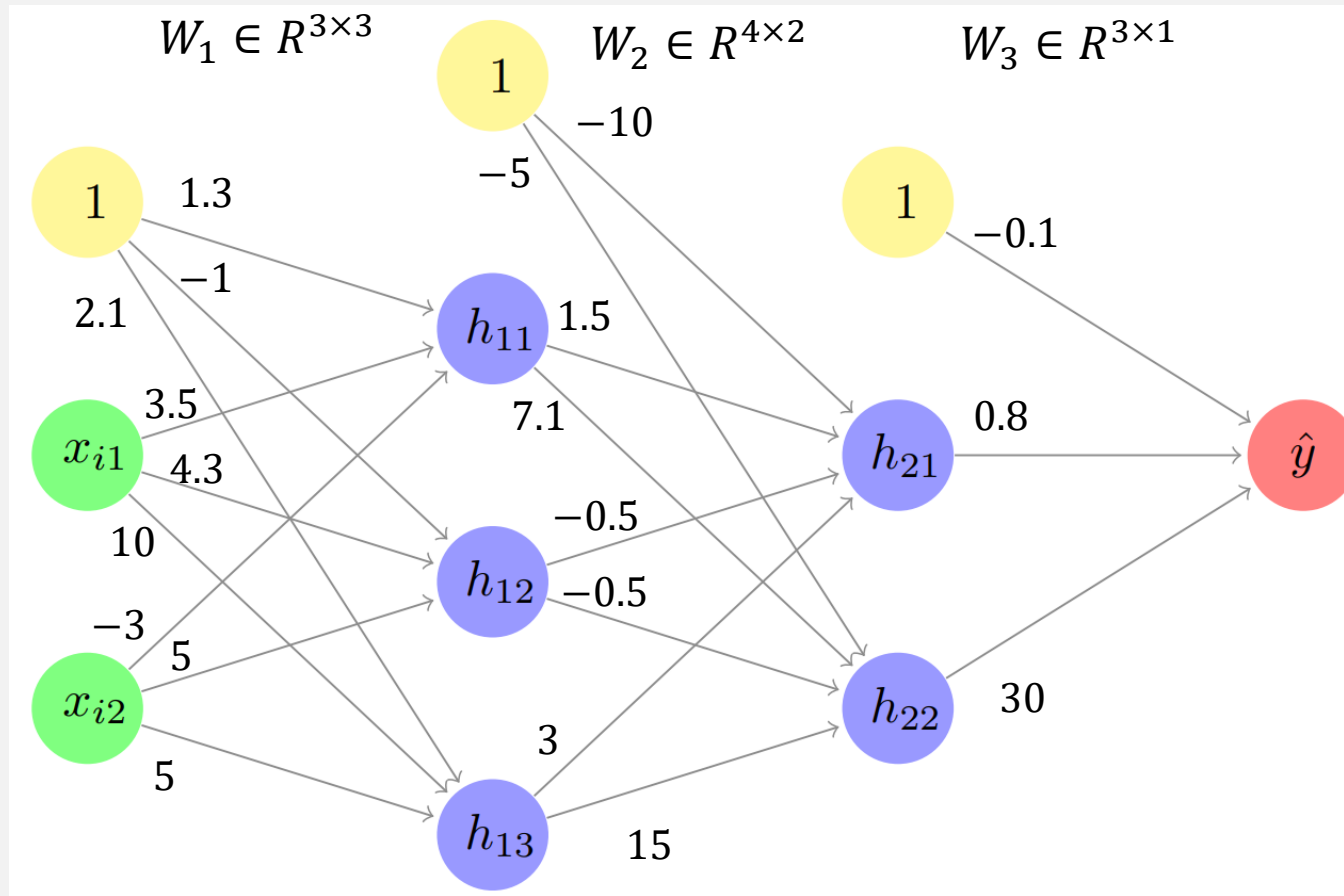
Multilayer NN [2]

- Each hidden unit takes in the weighted sum of the previous layer and performs an activation function.



- Using **non-linear** activation functions allows the network to approximate highly complex non-linear functions.

- Assuming all hidden units use sigmoid activation and the output predicts regression (linear activation), what is the prediction of $x_{new} = [1.1, -0.5]$?



$$h_{11} = \sigma((1 * 1.3) + (1.1 * 3.5) + (-0.5 * -3)) = 0.99$$

$$h_{12} = \sigma((1 * -1) + (1.1 * 4.3) + (-0.5 * 5)) = 0.77$$

$$h_{13} = \sigma((1 * 2.1) + (1.1 * 10) + (-0.5 * 5)) = 0.99$$

$$W_1 = \begin{bmatrix} 1.3 & -1 & 2.1 \\ 3.5 & 4.3 & 10 \\ -3 & 5 & 5 \end{bmatrix} \quad H_1 = \sigma(W_1^T x_i) = \begin{bmatrix} 0.99 \\ 0.77 \\ 0.99 \end{bmatrix}$$

$$h_{21} = \sigma((1 * -10) + (0.99 * 1.5) + (0.77 * -0.5) + (0.99 * 3)) = 0.003$$

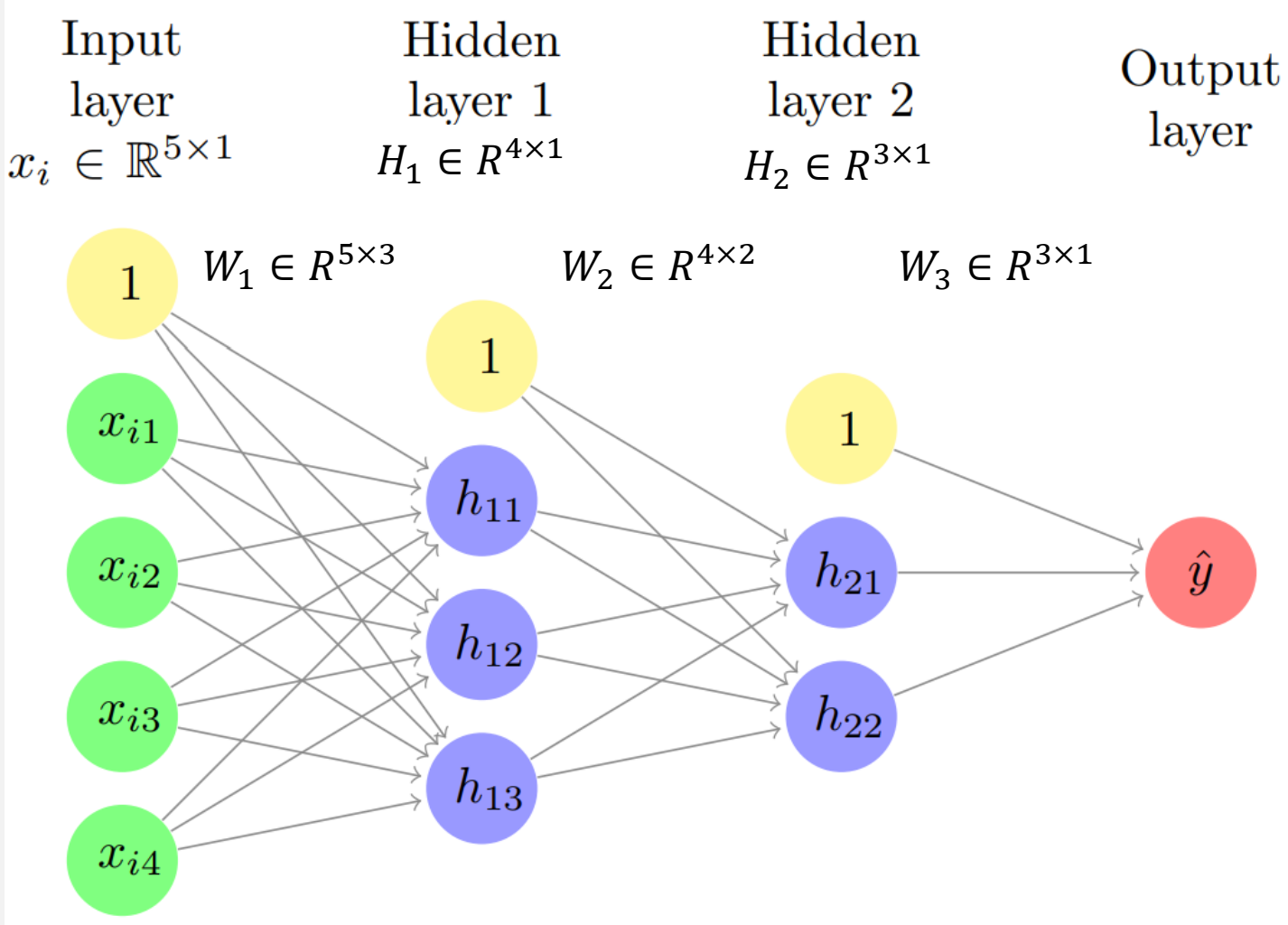
$$h_{22} = \sigma((1 * -5) + (0.99 * 7.1) + (0.77 * -0.5) + (0.99 * 15)) = 0.99$$

$$W_2 = \begin{bmatrix} -10 & -5 \\ 1.5 & 7.1 \\ -0.5 & -0.5 \\ 3 & 15 \end{bmatrix} \quad H_2 = \sigma(W_2^T H_1) = \begin{bmatrix} 0.003 \\ 0.99 \end{bmatrix}$$

$$\hat{y}_{new} = (1 * -0.1) + (0.003 * 0.8) + (0.99 * 30) = 29.60$$

$$W_3 = \begin{bmatrix} -0.1 \\ 0.8 \\ 30 \end{bmatrix} \quad \hat{y}_{new} = W_3^T H_2$$

Matrix view of Multilayer networks [1]



- The weights between layers can be represented as a matrix. In the example network on the left, we denote these weight matrices as W_1, W_2, W_3 .
- The size of a weight matrix is the number of previous layer units times the number of the next layer units.
- If we assume all hidden units apply a **sigmoid activation function**, and we predicting a regression label, then we get:

$$H_1 := \sigma(W_1^T x_i)$$

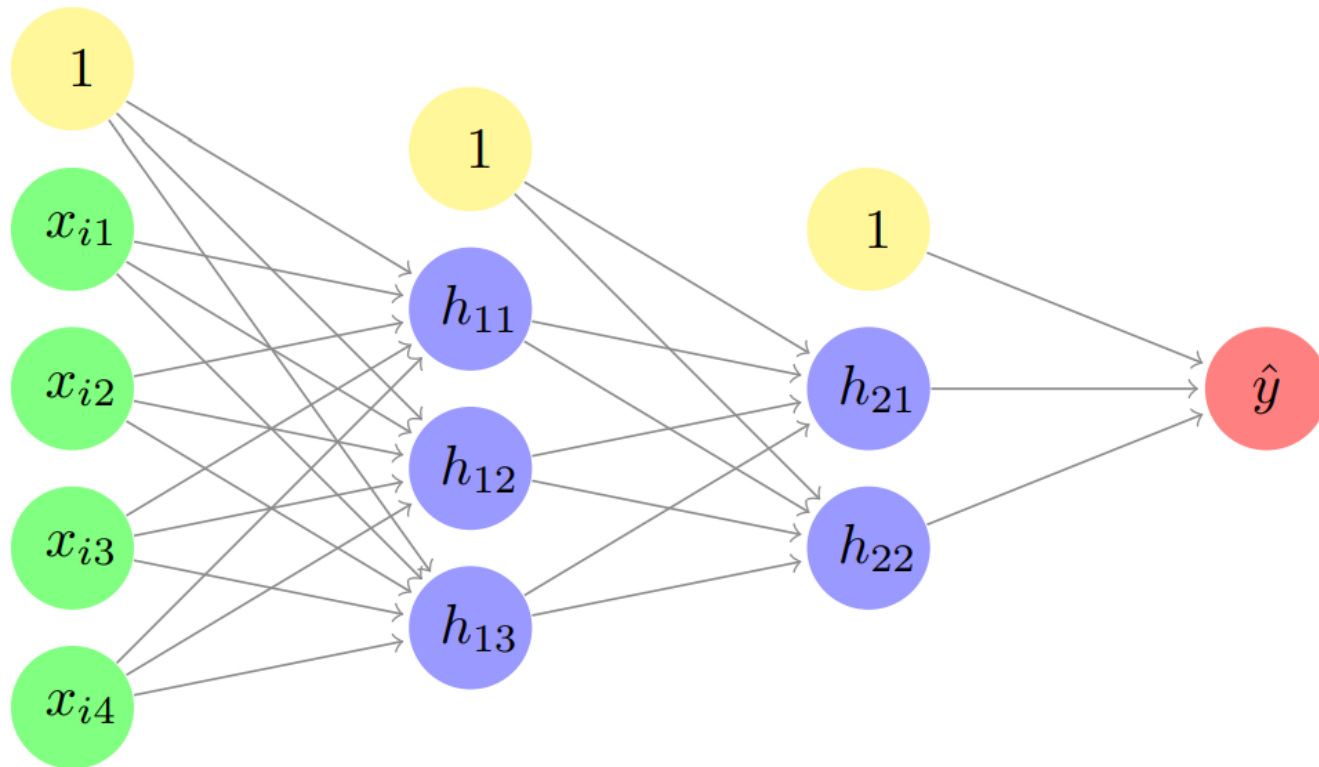
$$H_2 := \sigma(W_2^T H_1)$$

$$\hat{y} = W_3^T H_2$$

$$= W_3^T \sigma(W_2^T \sigma(W_1^T x_i))$$

Note we should add the bias for the hidden layer vectors

Importance of non-linear activations



- Thus, a neural network is technically doing matrix multiplication with element-wise activation functions in-between.
- **What if we assume all hidden units apply a **linear activation (identity)****, and we are predicting a regression label, then we get:

$$H_1 := W_1^T x_i$$

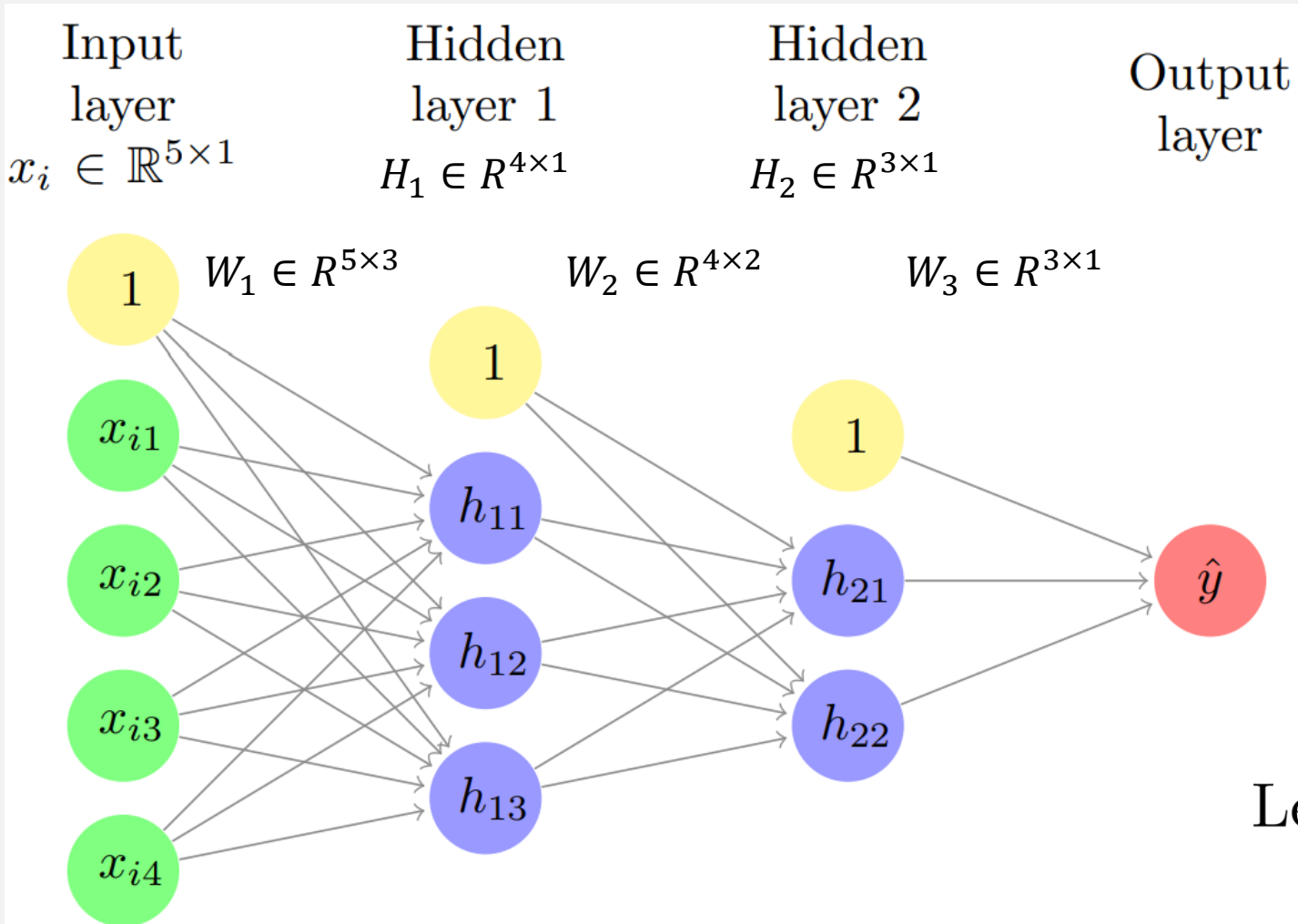
$$H_2 := W_2^T H_1$$

$$\begin{aligned}\hat{y} &= W_3^T H_2 \\ &= (W_3^T W_2^T W_1^T) x_i\end{aligned}$$

Let $A = (W_3^T W_2^T W_1^T)$ then $\hat{y} = Ax_i$

Note that this is similar to linear regression since A is just a weight vector.

Matrix view of Multilayer networks [2]



- Thus, a neural network is technically doing matrix multiplication with element-wise activation functions in-between.
- **What if we assume all hidden units apply a linear activation (identity)**, and we are predicting a regression label, then we get:

$$H_1 := W_1^T x_i$$

$$H_2 := W_2^T H_1$$

$$\begin{aligned}\hat{y} &= W_3^T H_2 \\ &= (W_3^T W_2^T W_1^T) x_i\end{aligned}$$

$$\text{Let } A = (W_3^T W_2^T W_1^T) \text{ then } \hat{y} = Ax_i$$

Note that this is similar to linear regression since A is just a weight vector.

Note2: The weight matrices as is cannot be multiplied because of their disagreeing dimension. We would need to account for the bias being added after each layer. We can add this to the weight matrices using *clever* but cumbersome notation. However, the end result would be the same; a vector A of weights.

Importance of non-linear activations

- We saw that multilayer networks are just a sequence of matrix multiplications of weight matrices. With just linear activations (identity), we get a simple linear regression model.
- Thus, using **non-linear** activation functions allows the network to approximate highly complex non-linear functions. The non-linear activations is what makes multilayer networks powerful.
- Recall in polynomial linear regression, we could approximate the sin function. But there we had to manually add polynomial features.
- In multilayer neural networks, we don't need to manually add polynomial features, we just need to add more hidden layers or more hidden units with non-linear activations. The input features stay the same!

Design choices in a multilayer network

- We have a supervised learning task (regression or classification), and we want to use a neural network. What are the design choices involved?

Architecture design:

1. **Input Layer:** These are the feature inputs. We can choose to add or remove the intercept/bias term.
2. **Hidden Layers:** The number of hidden layers to use. The number of hidden units in each hidden layer (these can be different per layer). The activation functions used (these can be different for each hidden unit). Whether or not to add an intercept/bias term for a hidden layer.
3. **Output Layer:** We can technically apply any activation for the output unit. Typically for regression a linear unit is used, and for classification a sigmoid unit is used.

Training a multilayer network [1]

- Given a training set and a multilayer network initialized with random weights, the goal is to find the weights that minimize a loss function.
- For neural networks in particular, gradient descent via **backpropagation** is used to minimize the loss function. We'll discuss this later.

$$\hat{w}_{ij} := \hat{w}_{ij} - \alpha \frac{\partial loss}{\partial w_{ij}}$$

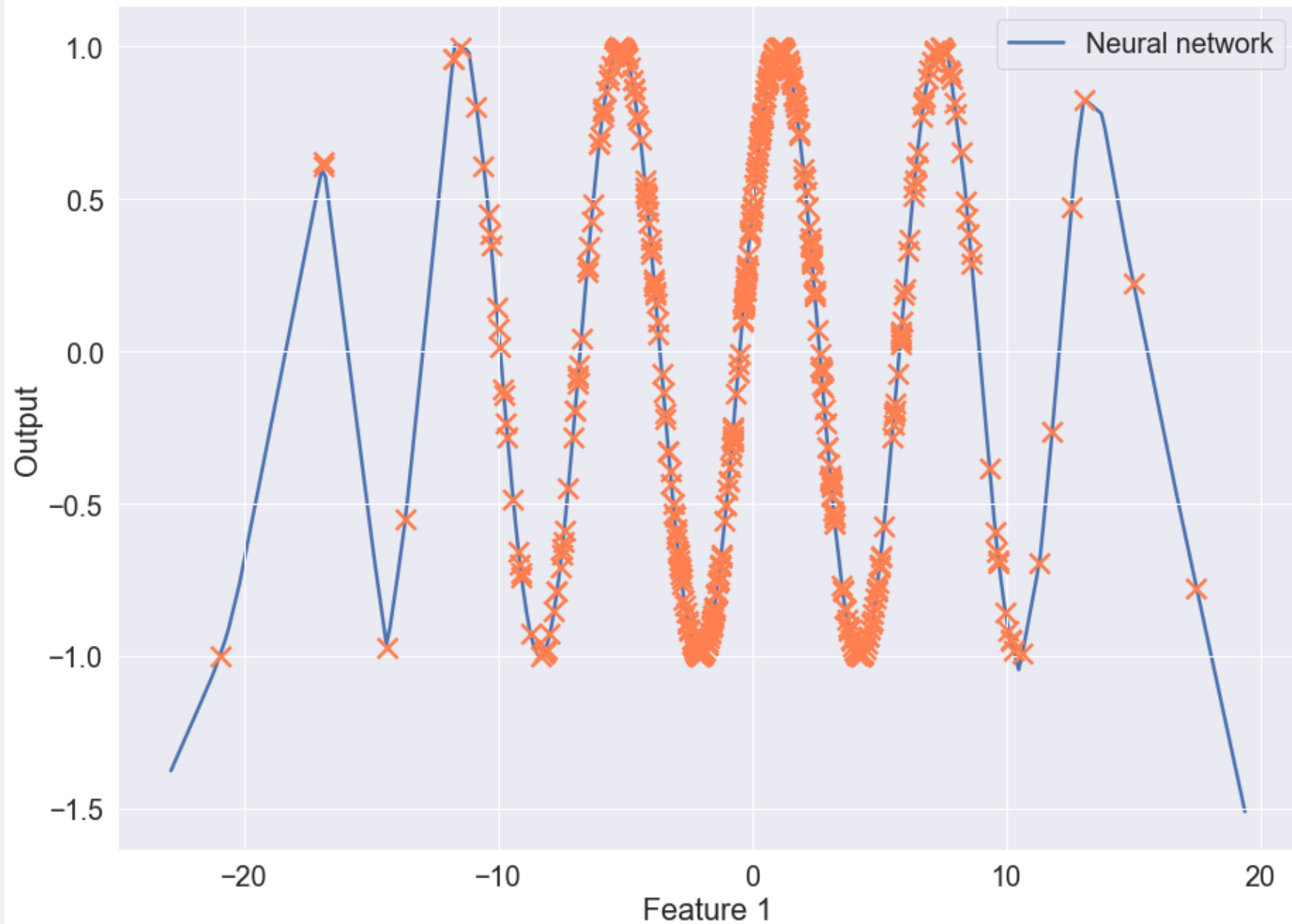
Regression loss: $loss(\hat{y}, y) = (\hat{y} - y)^2$

Binary classification: $loss(\hat{y}, y) = y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})$

- Also, you should Standard scale the features for stability during gradient descent.
- Finally, note that the use of non-linear activations leads to non-convex loss functions in a multilayer network. So, the weights that are found are most likely local minima.

NN in tensorflow [1]

- Recall this regression task.
- We can use the tensorflow python package to define a multilayer network.
- The network was then fit to the training data (orange points). tensorflow performs backpropagation to find the weights.
- tensorflow is a general purpose library for defining graph computations. For our purposes we will use its specific support for neural networks ([keras](#) module).



NN in tensorflow [2]

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers
4
5 model = keras.Sequential(
6     [
7         layers.Dense(1024, activation="relu", name="hidden_layer1", input_shape=(1,)),
8         layers.Dense(1024, activation="relu", name="hidden_layer2"),
9         layers.Dense(1, name="output"),
10    ]
11 )
12
13 opt = keras.optimizers.Adam(lr=0.001, amsgrad=True)
14 model.compile(optimizer=opt, loss='mse')
15 model.build()
16 model.summary()
```

- We defined a neural network with 2 hidden layers; first hidden layer has 1024 units using ReLu activation; second hidden layer has 1024 units using ReLu activation. Final layer is the output prediction with 1 linear unit.
- In line 13 we defined an [optimizer](#) that will be used in gradient descent. Adam is a popular choice here.
- Line 14 we compile the network with the optimizer and specify the loss. In this regression task we use mean squared error (mse).

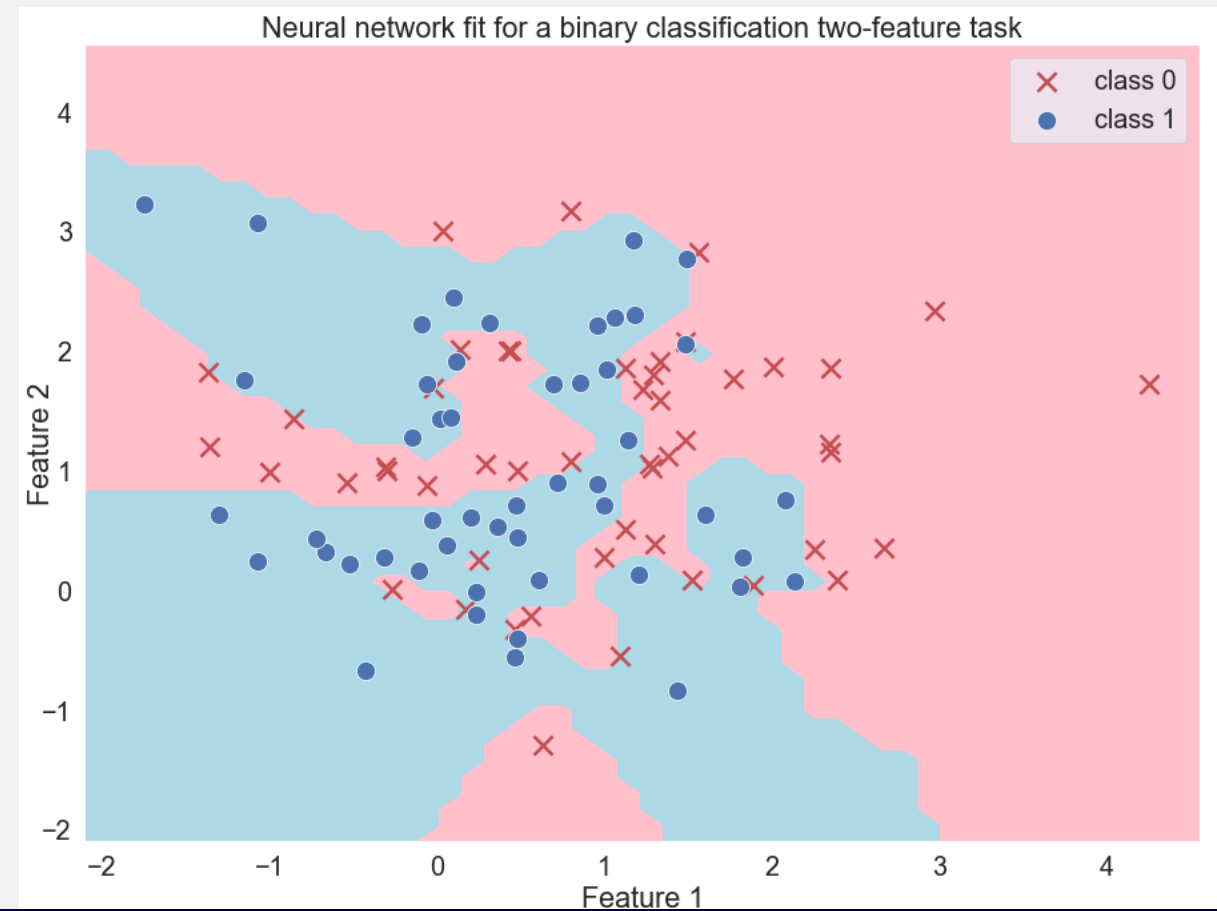
```
model.fit(X_train, y_train, batch_size=1024, verbose=2, epochs=10000)
```

- After defining and compiling the network, we can now start training by calling [fit](#).
- fit has many arguments, so it's worth to get familiar with it by reading the [documentation](#).
- The weights of the network are initialized randomly. The fit function then performs backpropagation on the training data to minimize the selected loss.
- Notice the argument batch_size. So keras is doing minibatch gradient descent.
- epochs is the number of iterations to run gradient descent.
- verbose defines the level of details for the logger.

```
Epoch 1/10000  
- 2s - loss: 0.8153  
Epoch 2/10000  
- 0s - loss: 203.5340  
Epoch 3/10000  
- 0s - loss: 25.6502  
Epoch 4/10000  
- 0s - loss: 21.7856  
Epoch 5/10000  
- 0s - loss: 27.7978  
Epoch 6/10000
```

NN in Tensorflow [4]

- For this binary classification task, we defined the network below.
- Notice that for the output unit we used a sigmoid activation to denote binary label prediction.
- See list of available activations in keras:
<https://keras.io/api/layers/activations/>



```
5 model = keras.Sequential(  
6     [  
7         layers.Dense(1024, activation="relu", name="hidden layer1", input_shape=(2,)),  
8         layers.Dense(1024, activation="relu", name="hidden layer2"),  
9         layers.Dense(1, activation="sigmoid", name="output"),  
0     ]  
1 )
```

```
opt = keras.optimizers.Adam(lr=0.001, amsgrad=True)
model.compile(optimizer=opt, loss='binary_crossentropy')
model.build()
model.summary()
|
model.fit(X_train, y_train, batch_size=1024, verbose=2, epochs=10000)
```

- Notice that the loss we used here is the binary crossentropy (logloss) for binary classification.
- Keras supports a variety of popular losses. See the documentation [here](#).

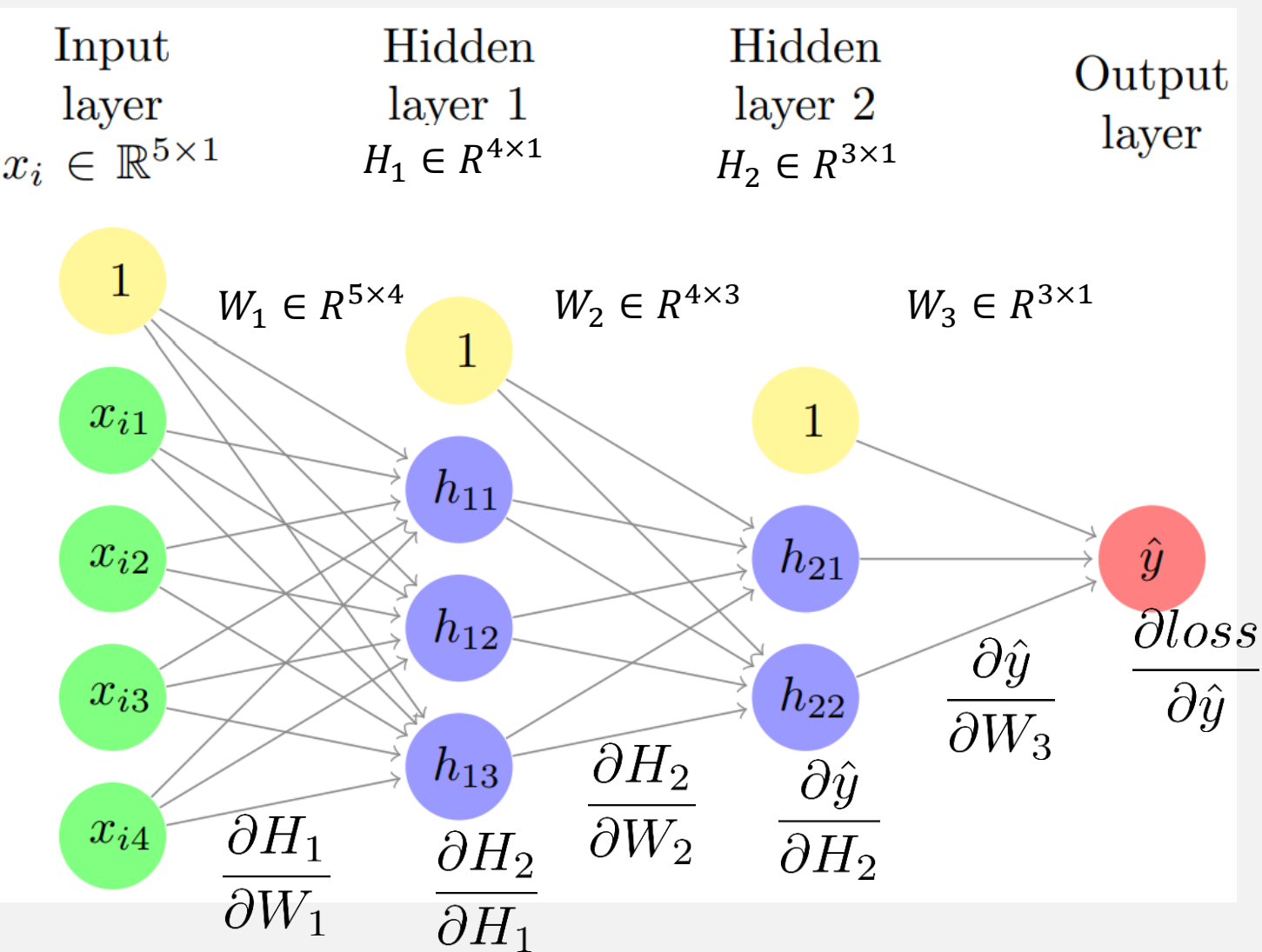
- After fitting the network. You can perform predictions as follow:

```
y_pred_on_test = model.predict(X_test)
```

- Popular python packages: tensorflow, keras (now incorporated into tensorflow), pytorch.
- Supports GPU devices. Underneath these packages there are GPU optimizations to speedup computations.
- Simple to implement a complex neural network using these packages. These packages can even support custom user-defined operations.

Quick Overview of Backpropagation

Multilayer NN Backpropagation: vectorized notation



Order of backward gradient computation:

$$\frac{\partial \text{loss}}{\partial \hat{y}}$$

$$\frac{\partial \text{loss}}{\partial W_3} = \frac{\partial \hat{y}}{\partial W_3} \frac{\partial \text{loss}}{\partial \hat{y}}$$

$$\frac{\partial \text{loss}}{\partial H_2} = \frac{\partial \hat{y}}{\partial H_2} \frac{\partial \text{loss}}{\partial \hat{y}}$$

$$\frac{\partial \text{loss}}{\partial W_2} = \frac{\partial H_2}{\partial W_2} \frac{\partial \text{loss}}{\partial H_2}$$

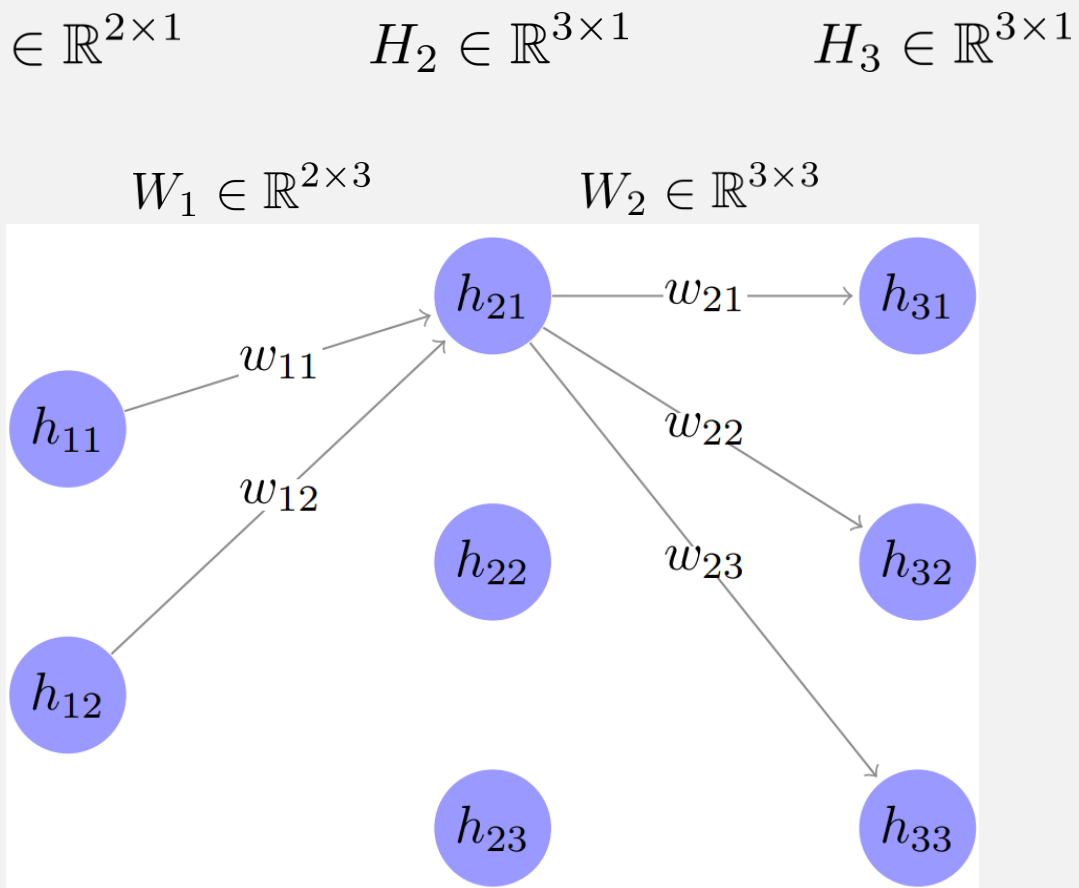
$$\frac{\partial \text{loss}}{\partial H_1} = \frac{\partial H_2}{\partial H_1} \frac{\partial \text{loss}}{\partial H_2}$$

$$\frac{\partial \text{loss}}{\partial W_1} = \frac{\partial H_1}{\partial W_1} \frac{\partial \text{loss}}{\partial H_1}$$

Reuse
computed
values

$$\hat{W}_j := \hat{W}_j - \alpha \frac{\partial \text{loss}}{\partial W_j} \text{ for } j = 1, 2, 3$$

Multilayer NN Backpropagation: hidden node [1]



We have randomly initialized the weights, and forward passed an instance. For the hidden node h_{21} , the backpropagation is:

$$\frac{\partial loss}{\partial H_3} = \begin{bmatrix} \frac{\partial loss}{\partial h_{31}} \\ \frac{\partial loss}{\partial h_{32}} \\ \frac{\partial loss}{\partial h_{33}} \end{bmatrix}$$

Assume this has already been computed. That is, this is a vector of values.

The above values can be **reused** to compute partial derivatives in previous layer:

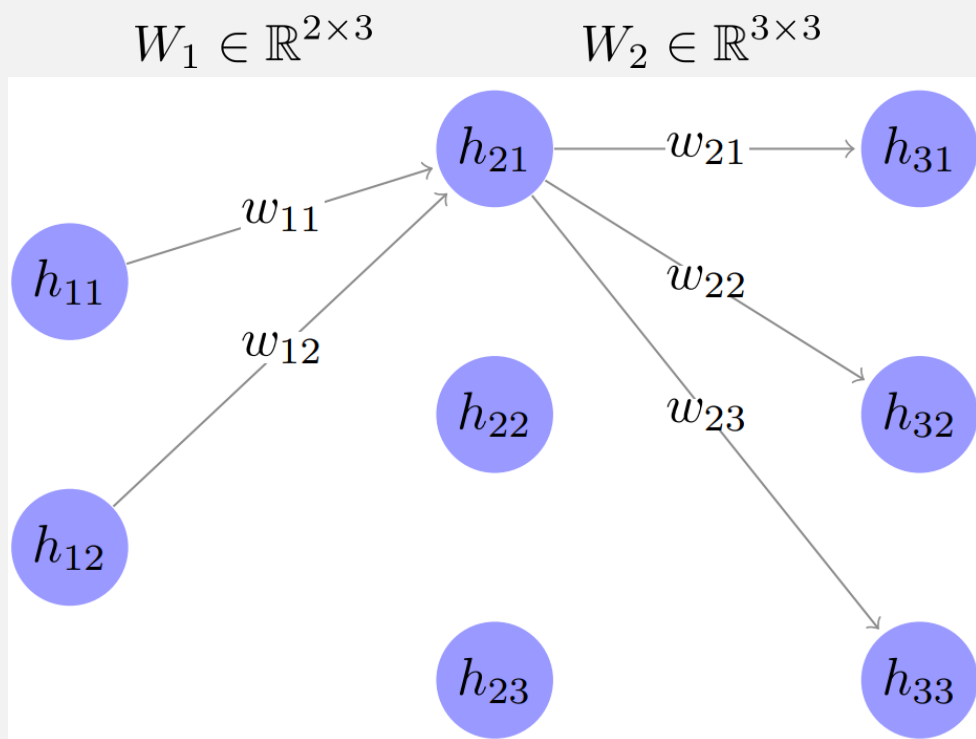
$$\frac{\partial loss}{\partial w_{21}} = \frac{\partial h_{31}}{\partial w_{21}} \frac{\partial loss}{\partial h_{31}} \implies \hat{w}_{21} := \hat{w}_{21} - \alpha \frac{\partial loss}{\partial w_{21}}$$

$$\frac{\partial loss}{\partial w_{22}} = \frac{\partial h_{32}}{\partial w_{22}} \frac{\partial loss}{\partial h_{32}} \implies \hat{w}_{22} := \hat{w}_{22} - \alpha \frac{\partial loss}{\partial w_{22}}$$

$$\frac{\partial loss}{\partial w_{23}} = \frac{\partial h_{33}}{\partial w_{23}} \frac{\partial loss}{\partial h_{33}} \implies \hat{w}_{23} := \hat{w}_{23} - \alpha \frac{\partial loss}{\partial w_{23}}$$

Multilayer NN Backpropagation: hidden node [2]

$$H_1 \in \mathbb{R}^{2 \times 1} \quad H_2 \in \mathbb{R}^{3 \times 1} \quad H_3 \in \mathbb{R}^{3 \times 1}$$



The partial derivative at the node h_{21} is:

$$\frac{\partial loss}{\partial h_{21}} = \frac{\partial H_3}{\partial h_{21}} \frac{\partial loss}{\partial H_3} = \left[\begin{array}{c} \frac{\partial g(h_{21}w_{21} + \dots + h_{23}w_{27})}{\partial h_{21}} \\ \frac{\partial g(h_{21}w_{22} + \dots + h_{23}w_{28})}{\partial h_{21}} \\ \frac{\partial g(h_{21}w_{23} + \dots + h_{23}w_{29})}{\partial h_{21}} \end{array} \right]^T \frac{\partial loss}{\partial H_3}$$

$$= \begin{bmatrix} g'(h_{31})w_{21} & g'(h_{32})w_{22} & g'(h_{33})w_{23} \end{bmatrix} \left[\begin{array}{c} \frac{\partial loss}{\partial h_{31}} \\ \frac{\partial loss}{\partial h_{32}} \\ \frac{\partial loss}{\partial h_{33}} \end{array} \right]$$

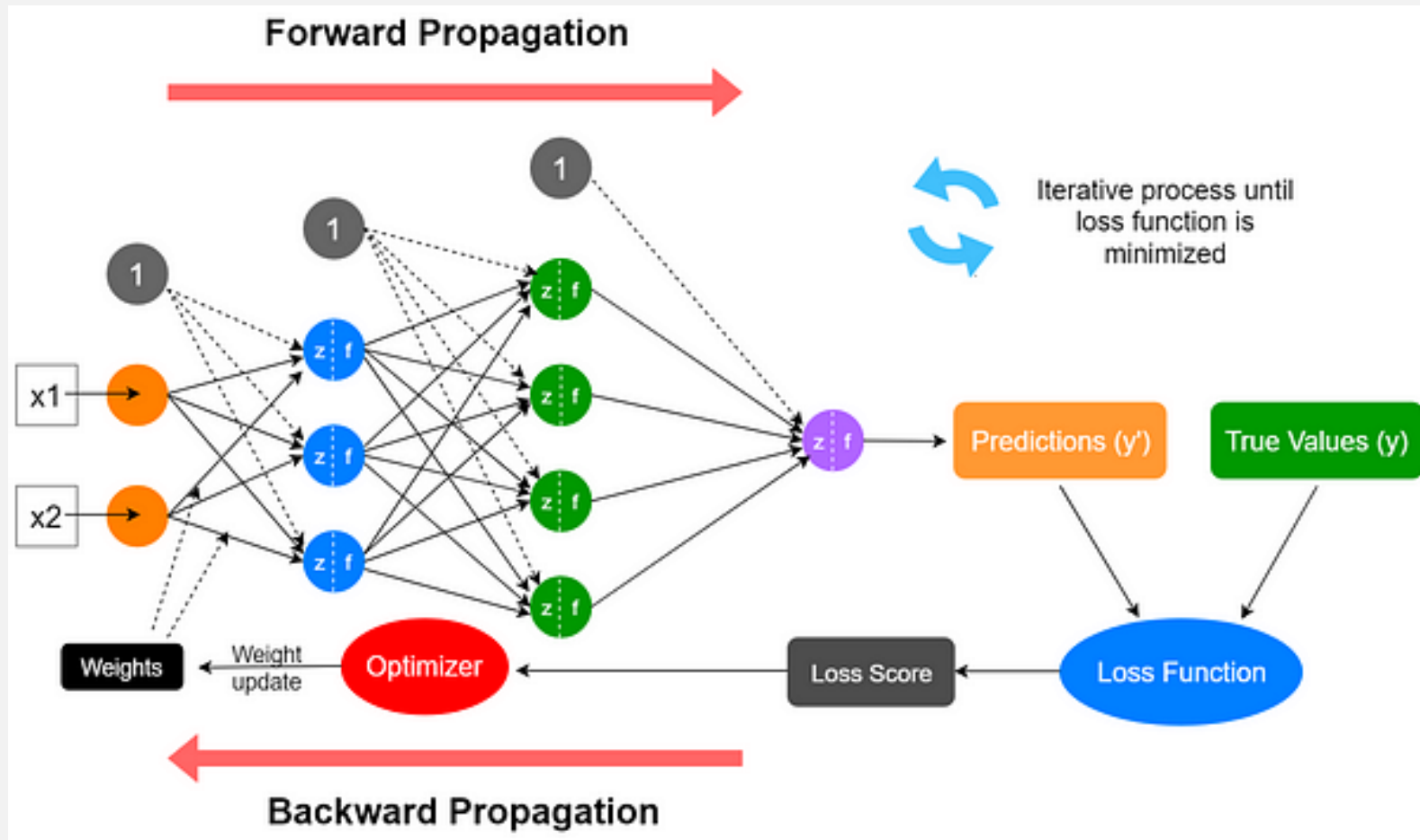
So, the derivative at a node is a weighted sum of the derivatives at the next layer. Also, we can **reuse**:

$$\frac{\partial loss}{\partial w_{11}} = \frac{\partial h_{21}}{\partial w_{11}} \frac{\partial loss}{\partial h_{21}} \implies \hat{w}_{11} := \hat{w}_{11} - \alpha \frac{\partial loss}{\partial w_{11}}$$

$$\frac{\partial loss}{\partial w_{12}} = \frac{\partial h_{21}}{\partial w_{12}} \frac{\partial loss}{\partial h_{21}} \implies \hat{w}_{12} := \hat{w}_{12} - \alpha \frac{\partial loss}{\partial w_{12}}$$

Training a multilayer network Summary

- Given a training set and a multilayer network initialized with random weights, the goal is to find the weights that minimize a loss function.
- The process of taking an instance, doing a **forward pass**, computing the gradient from the output layer to the input layer, and updating the weights is called **backpropagation**.
- The main takeaway is that backpropagation gives us a systematic way of updating the weights and reusing gradient computations. The main technique used is chain-rule from calculus.
- Packages like pytorch/keras handle computing the gradients and updating the network weights. So, the user just defines the network architecture and loss function, then the package will handle training the weights of the network.
- For those interested in more details, see [link](#) and [link](#)



Multilayer network stability: non-convexity of the loss [1]

- In multilayer networks with non-linear activations, the loss is almost surely non-convex.
- This means that the initial weights we set have an effect on where we start in the loss curve.
- In other words, if we randomly initialize the weights, then run gradient descent for a while, we can reach a local minima. If we do this again, we may end up in another local minima.
- Also, you should Standard scale the features for stability.

Multilayer network stability: non-convexity of the loss [2]

- Consider the toy visualization of the non-convex loss curve to the right.
- Depending on where we start with the initial weights, we may end up at different local minima.
- This is illustrated by the 3 paths on the plot.
- Despite this, we still take the local minima solution.

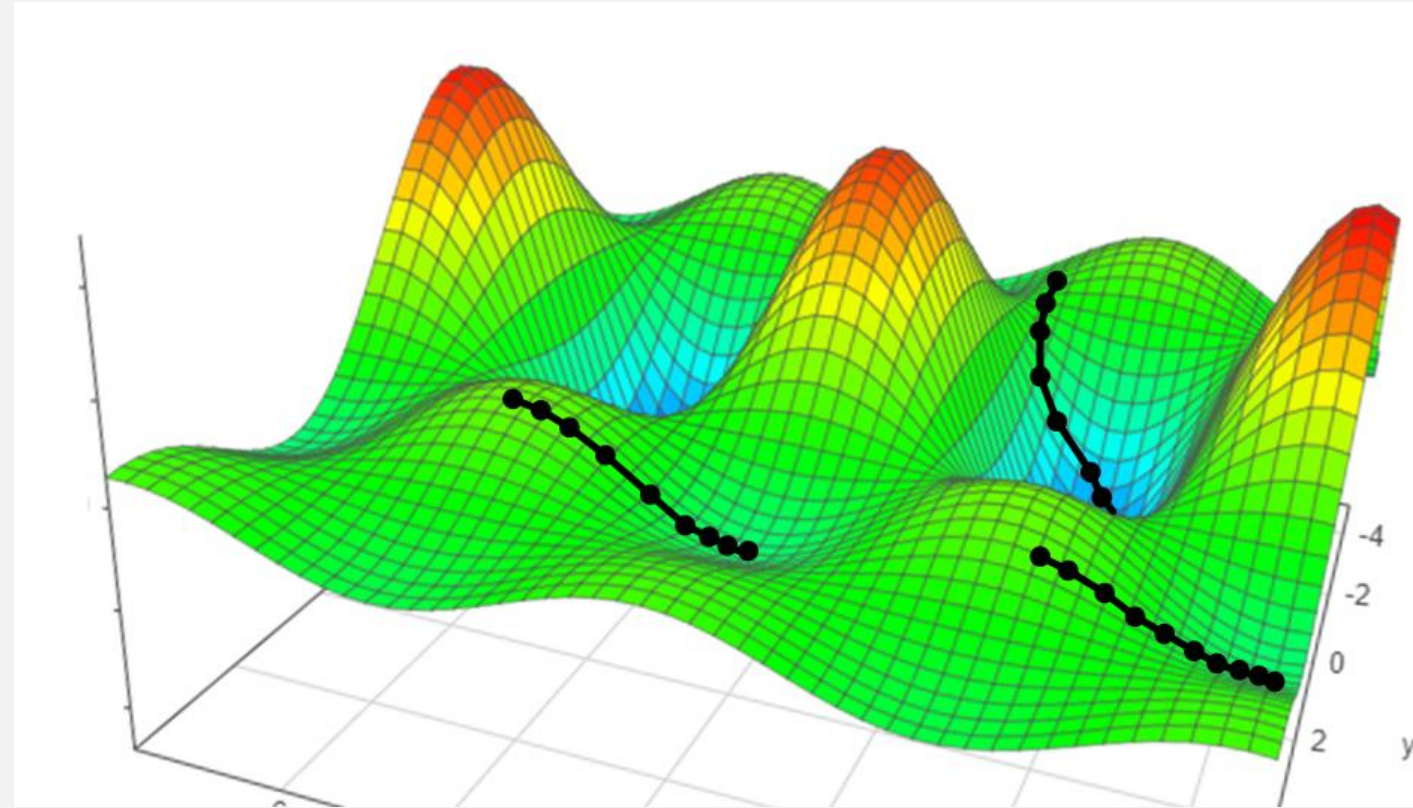


Image from: [link](#)

Multilayer network stability: non-convexity of the loss [2]

- Minibatch gradient descent **may** help us get out of local minima.
- This is because a single batch makes a *noisy* updates.
- From studies in neural networks, minibatch gd converges faster than full-batch GD, and *generally* reaches a better local minima. Note this is not a fixed rule that always happens.

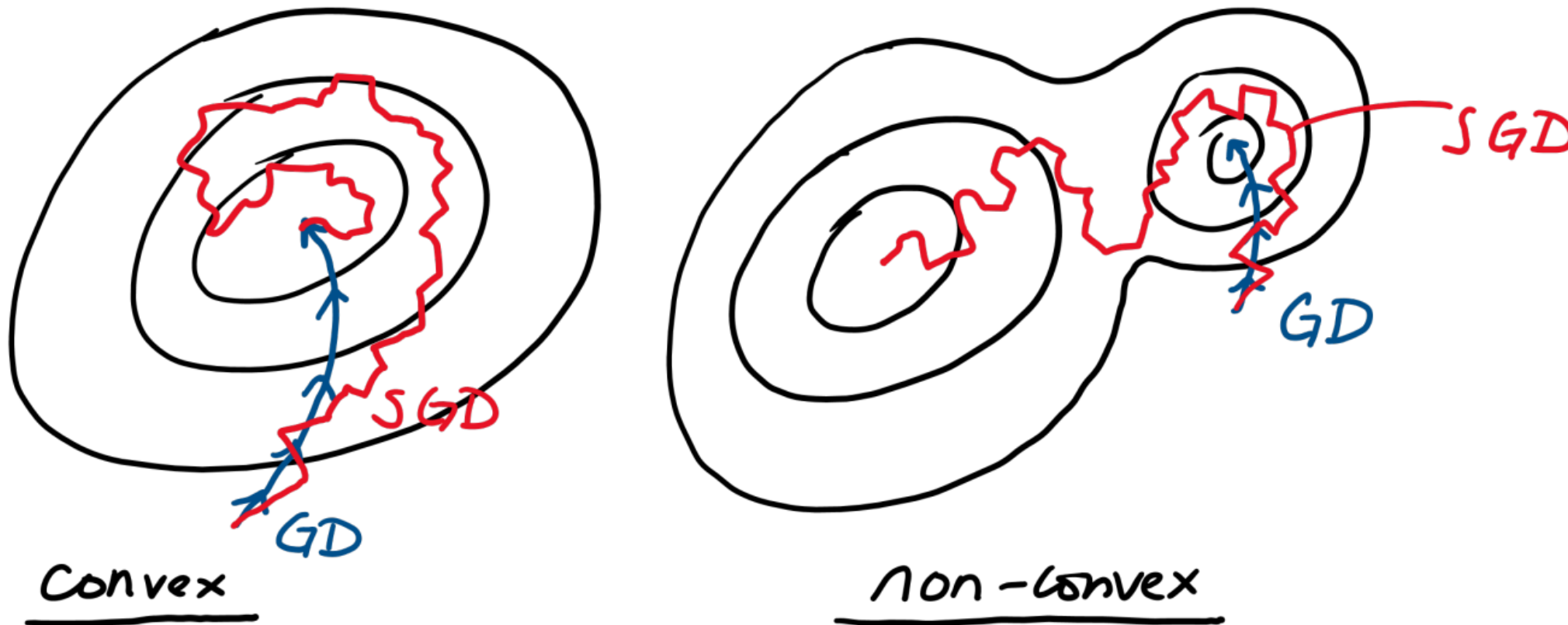


Image from: [link](#)

Combating Overfitting via Early Stopping

Overfitting in NN [1]

- Multilayer neural networks are powerful. Given any function, you can construct a 2 hidden layer network that can approximate it.
- Unfortunately, this means that given a training set, you can construct a multilayer network that overfits it easily.
- The complexity or power of a neural network is determined by its architecture: number of hidden layers, number of hidden units, and activation functions.
- Thus, it is preferred to use a *simpler* network that fits the training data over a more *complex* network.

Overfitting in NN [2]

- Also recall that the loss function in a multilayer network with non-linear activations is almost surely non-convex.
- The loss surface is a function of the training data, network architecture, and weights.
- Given the training data and network architecture, we can combat overfitting **during** training by early stopping.
- **Early stopping:** Compute the performance on a validation set while running gradient descent on training set. When the validation and training performance start to diverge too much, then stop running gradient descent.

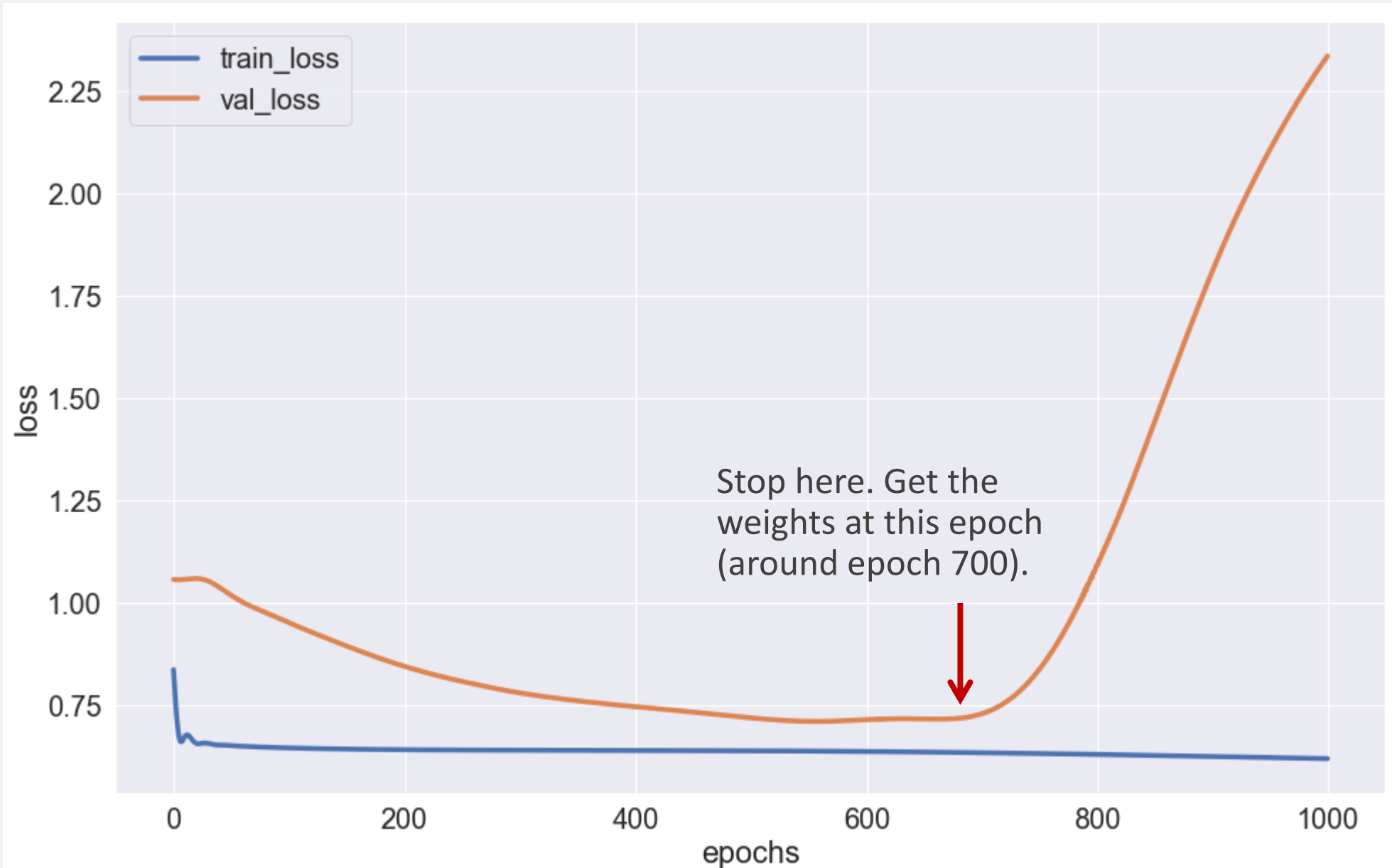
Early stopping [1]

- Given a training set, validation set, and a neural network with randomly initialized weights. Do the following:

For iterations $i = 1, 2, \dots$ do:

1. Update the network weights using backpropagation on the training data.
2. Compute the performance on the validation set and training set.
3. Stop iterating when the validation and training performance diverge. In other words, when the difference between the performances gets too large.

Early stopping [2]



Early stopping [3]

- In keras, you can monitor the validation loss while fitting by passing the validation data:

```
logs = model.fit(X_train, y_train, validation_data=(X_val, y_val), batch_size=1024, verbose=2, epochs=1000)
```

```
- 0s - loss: 0.6401 - val_loss: 0.8579
```

```
Epoch 186/1000
```

```
- 0s - loss: 0.6401 - val_loss: 0.8569
```

```
Epoch 187/1000
```

```
- 0s - loss: 0.6401 - val_loss: 0.8560
```

```
Epoch 188/1000
```

```
- 0s - loss: 0.6400 - val_loss: 0.8550
```

```
Epoch 189/1000
```

```
- 0s - loss: 0.6400 - val_loss: 0.8541
```

```
Epoch 190/1000
```

Early stopping [4]

- In keras, you can monitor the validation loss and perform early stopping by using [callbacks](#):

```
1 earlystop_callback = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3)
2
3 logs = model.fit(X_train, y_train, validation_data=(X_val, y_val),
4                 callbacks=[earlystop_callback],
5                 batch_size=1024, verbose=2, epochs=1000)
```

- Training will be stopped when val_loss does not increase for more than 3 epochs. You can alter this patience parameter in the callback.
- logs contains the training history. You can get each epoch training loss with logs.history["loss"] or validation loss with logs.history["val_loss"].

Overfitting in NN Summary

- To summarize, it is easy to construct a network that overfits the training data. Particularly if the training data is small.
- To combat overfitting, you have the following options:
 1. Get more data!
 2. Use a *simpler* network architecture. Less hidden layers, less hidden units.
 3. Early stop the training using a validation set.

Brief Overview of CNNs

Image Classification [1]

- Yes, we can use machine learning with an image as input, and a yes/no label to indicate that the image has the template.
- Example task: detect if an image has a camel.
 - Input: 300x300 grayscale image.
 - Label: yes/no binary label.
 - Model: neural network that outputs probability of the input image containing a camel.

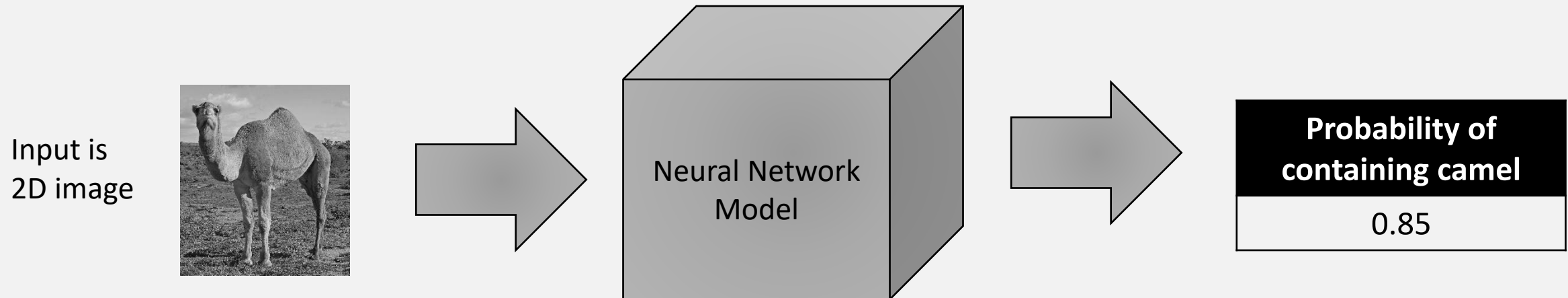
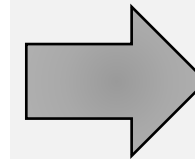
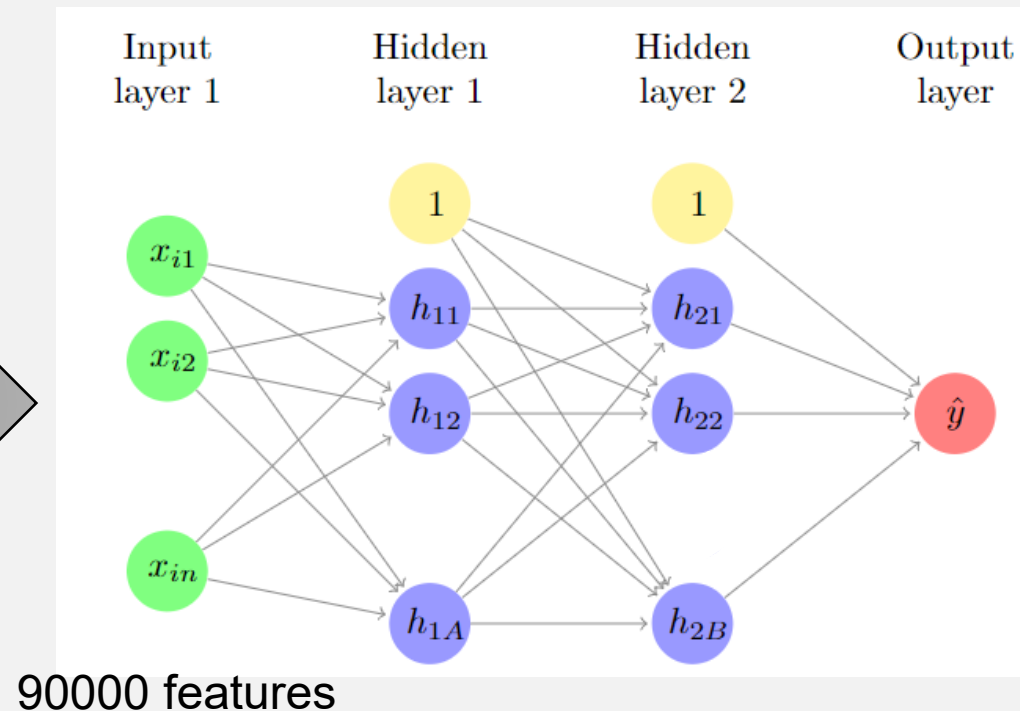
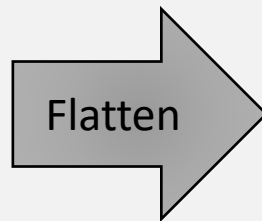


Image Classification [2]

- Neural network architecture #1: flatten the image and use a fully-connected neural network.
- Disadvantages of this approach:
 - Large number of weight parameters to be learned. Increases model complexity, and as a consequence, requires more data to learn a good model.
 - More importantly, image pixels have **local** spatial-dependence, that is one pixel is related to its **nearby** surrounding pixels. It might be better to learn local relationships in an image.



Input is
2D image
300x300

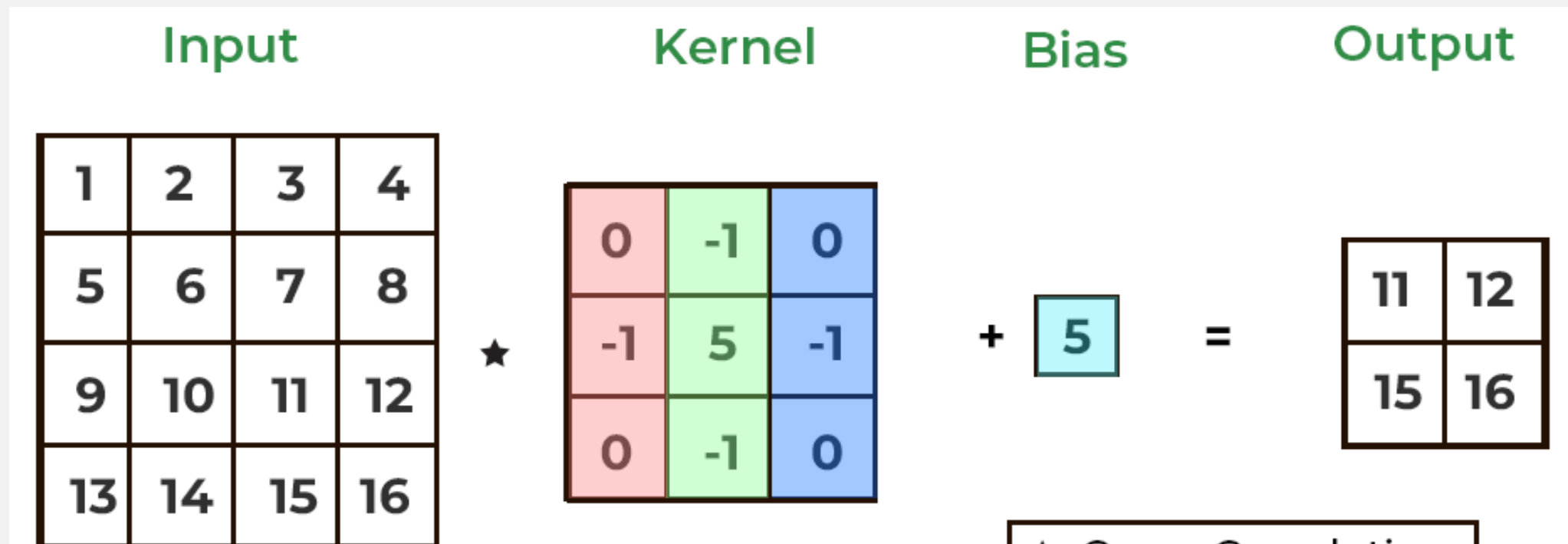


Probability of
containing camel

0.85

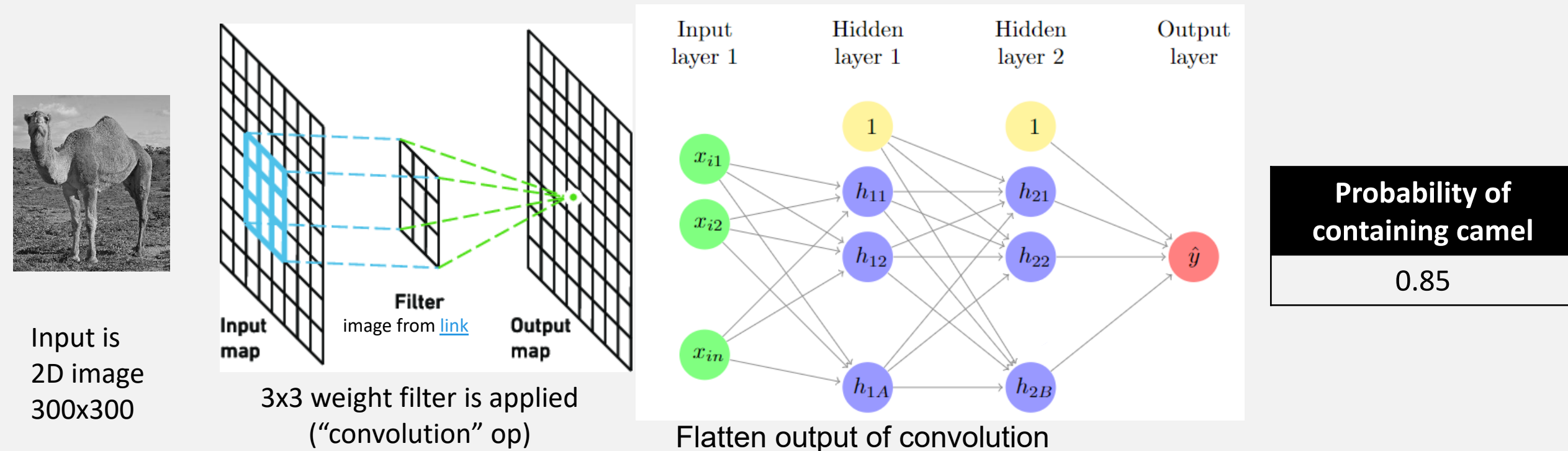
Convolution Operation

- A convolution operation involves an input image and a **kernel** (also called **filter**).
- The kernel is applied over the input image elementwise and summed up as shown below.
- A bias can also be used (or not).
- The kernel is trying to detect the presence of a pattern in the input image. This detection is represented at the output.
- We want to use this idea of kernels to detect the presence of local-spatial *patterns* in the image indicating the object we are interested in.



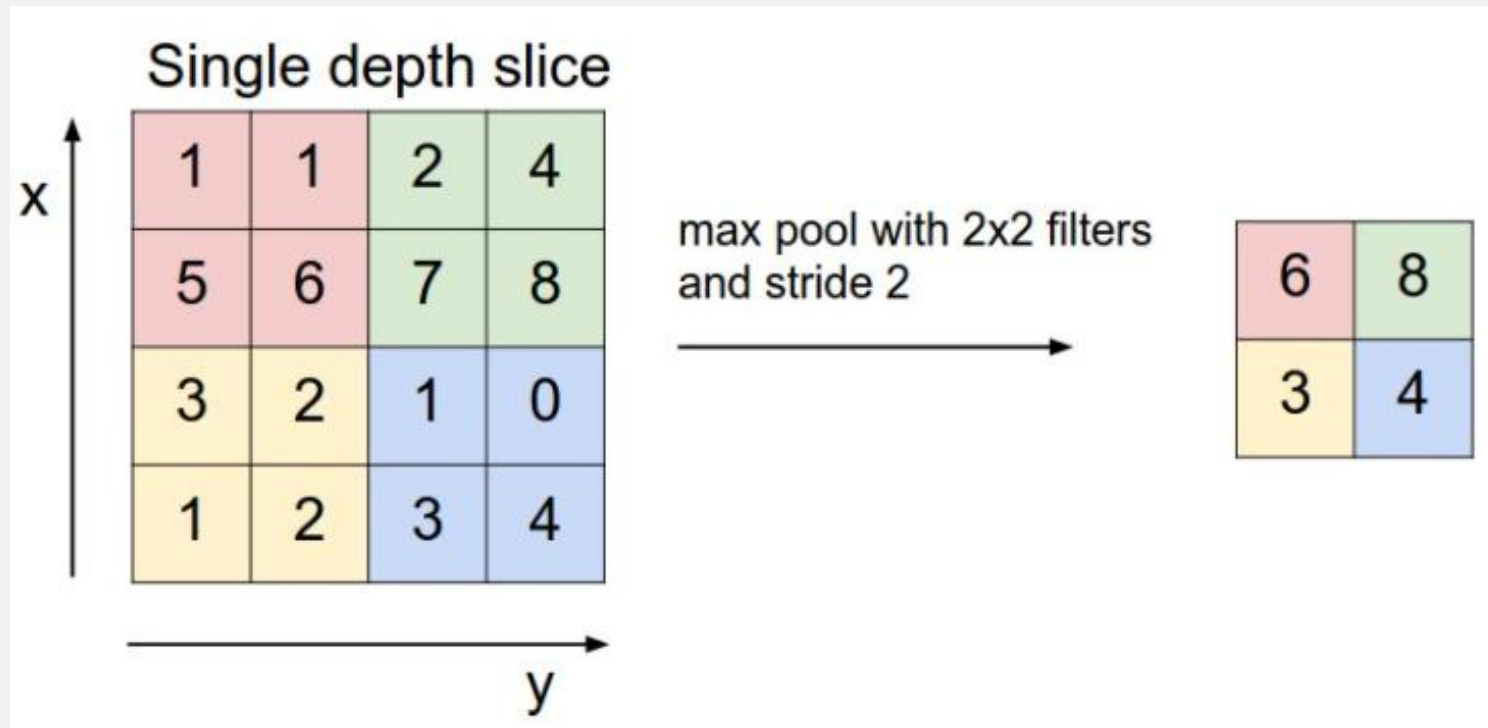
Convolutions in Neural Network

- Neural network architecture #1: apply convolutions with weight filters/kernel. Then flatten the resulting image, then apply a fully-connected network.
- Advantages of this approach: with convolutions, relationship among local pixels is learned.
- Disadvantage: a single filter may not be enough to learn the target concept. Furthermore, no considerable reduction in image size which still leads to large number of parameters in the FCN layers.

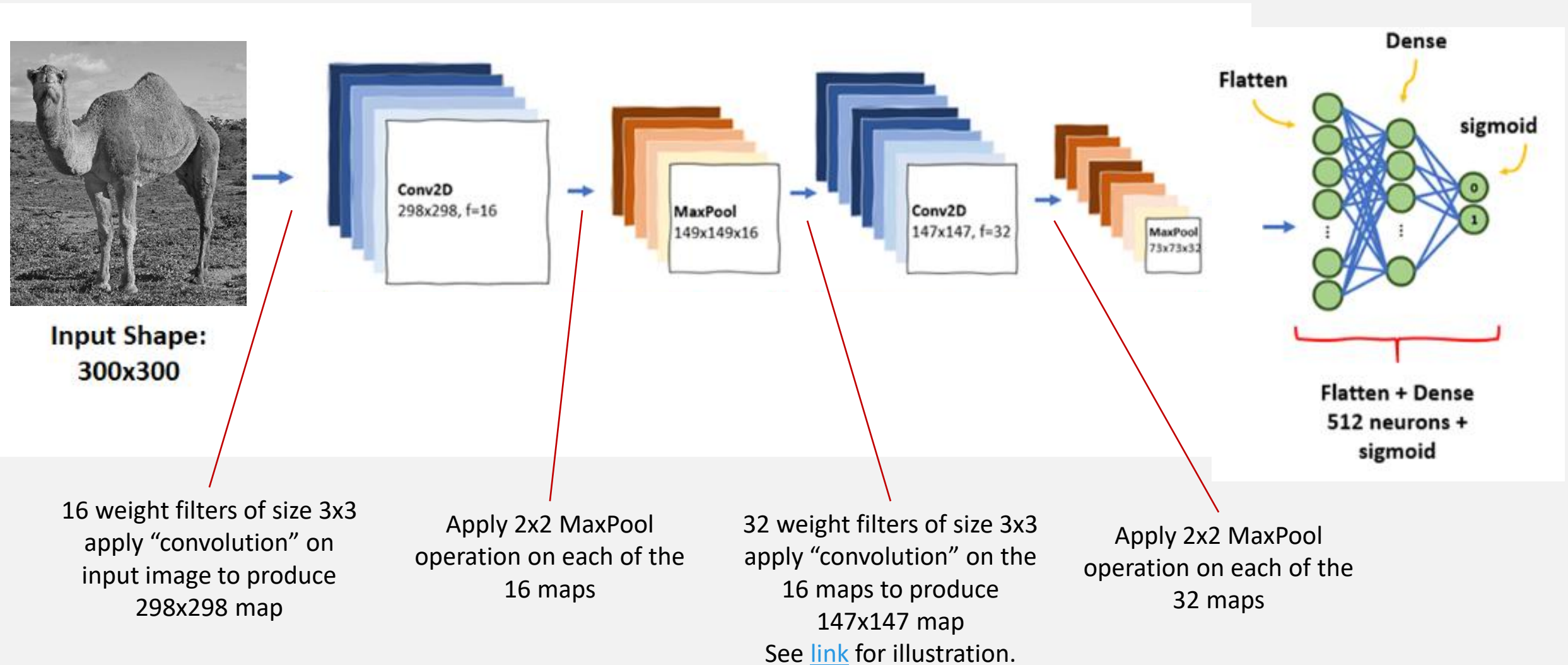


Pooling layers to reduce size

- Many types of summarizing operations; one common method is called Max-Pool.



CNN Architecture Example



CNN in Keras

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
```

```
model = tf.keras.models.Sequential([
```

```
# This is the first convolution, input_shape= 300x300 image
tf.keras.layers.Conv2D(16, (3,3), activation='relu', input_shape=(300, 300, 1)),
tf.keras.layers.MaxPooling2D(2, 2),
```

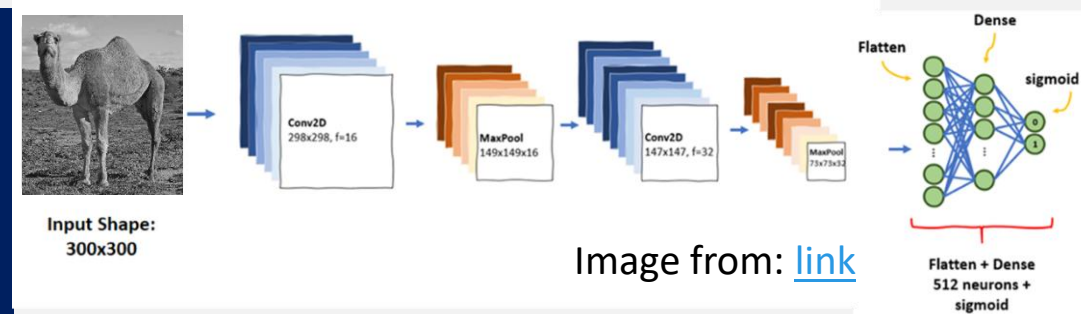
```
# The second convolution
tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
tf.keras.layers.MaxPooling2D(2,2),
```

```
# flatten layer then connect FCN layers
tf.keras.layers.Flatten(),
tf.keras.layers.Dense(512, activation='relu'), # 512 neuron hidden layer
# binary classification output
tf.keras.layers.Dense(1, activation='sigmoid')
```

```
])
```

```
# to get the summary of the model
print(model.summary())
```

```
# configure the model for training by adding metrics
model.compile(loss='binary_crossentropy', optimizer=Adam(lr=0.001), metrics=['accuracy'])
```



Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 298, 298, 16)	160
max_pooling2d (MaxPooling2D)	(None, 149, 149, 16)	0
conv2d_1 (Conv2D)	(None, 147, 147, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 73, 73, 32)	0
flatten (Flatten)	(None, 170528)	0
dense (Dense)	(None, 512)	87310848
dense_1 (Dense)	(None, 1)	513
Total params: 87,316,161		
Trainable params: 87,316,161		
Non-trainable params: 0		
None		

CNN Image Classification Example

- The intermediate maps are also 2D images that can be plotted. However, it can be difficult to interpret them.
- See below a CNN trained to predict a multi-class task (car, truck, airplane, ship, horse).
- Notice the final POOL layer is sparse; i.e. few highly active pixels.

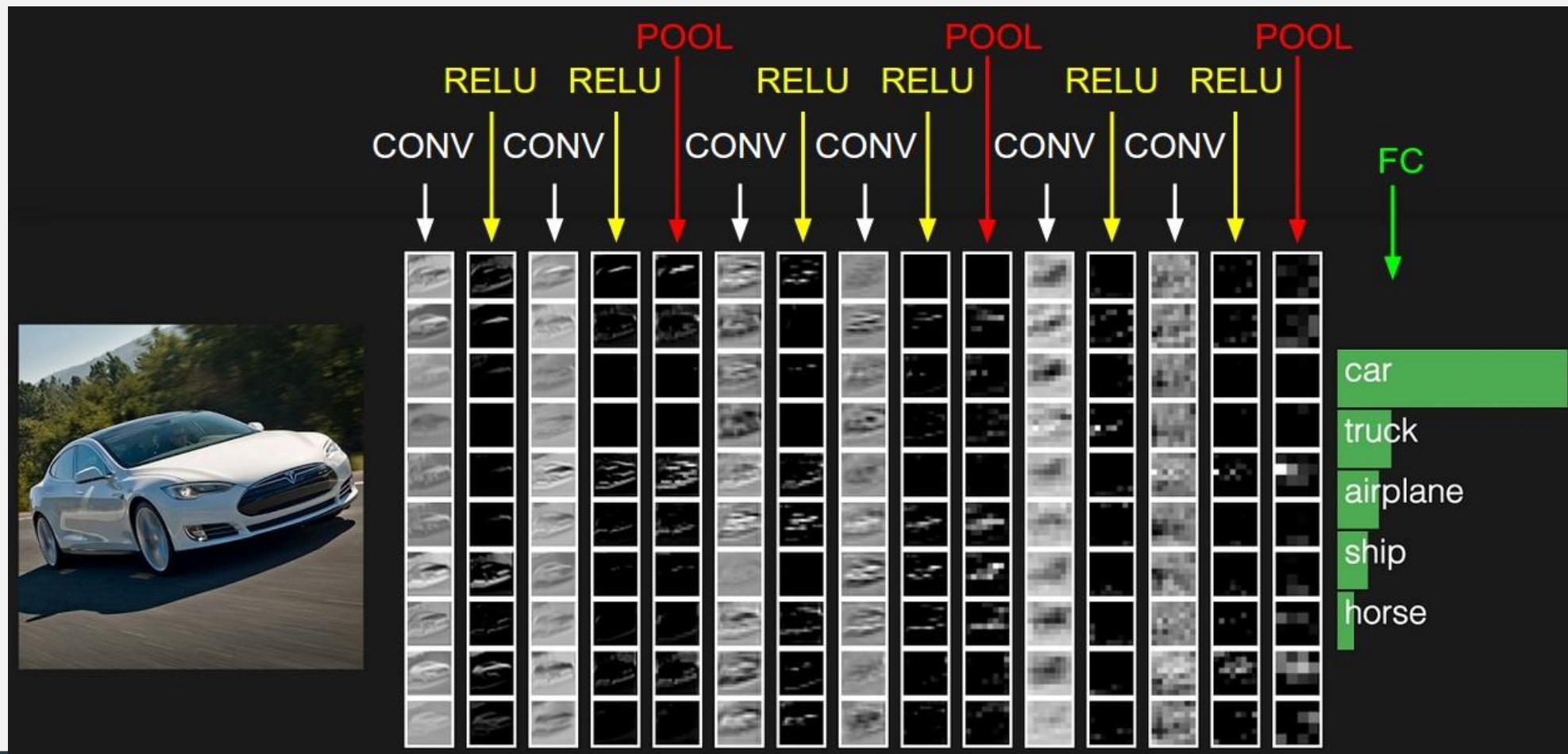


Image from: [link](#)